

Compiler Front End

- The front end of a compiler **analyzes a source program** and **creates an intermediate representation (IR, 中间表示)**, from which the back end generates target code
- Ideally, details of the source language are confined to the front end, and details of the target machine to the back end

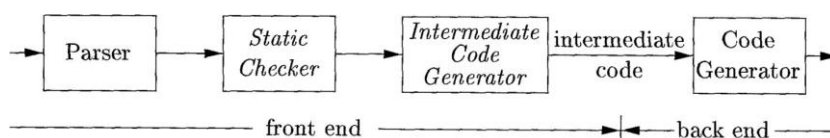


Figure 6.1: Logical structure of a compiler front end

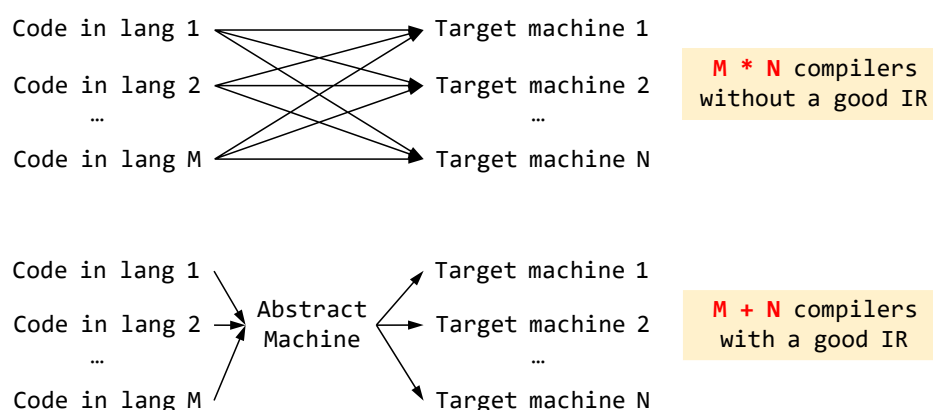
在编译器的分析——综合模型中，前端对源程序进行分析并产生中间表示，后端在此基础上生成目标代码。理想情况下，和源语言相关的细节在前端分析中处理，即关于目标机器器的细节则在后端处理。

本章的内容涉及中间代码表示、静态类型检查和中间代码生成。为简单起见，我们假设一个编译程序的前端处理按照图 6-1 所示方式进行组织，顺序地进行语法分析、静态检查和中间代码生成。有时候这几个过程也可以结合起来，在语法分析中一并完成。我们将使用第 2 章和第 5 章中的语法制导定义来描述类型检查和翻译过程。大部分的翻译方案可以基于第 5 章中给出的自顶向下或自底向上的语法分析技术来实现。所有的方案都可以通过生成并遍历抽象语法树来实现。

静态检查包括类型检查（type checking），类型检查保证运算被应用到兼容的运算分量。静态检查还包括在语法分析之后进行的所有语法检查。例如，静态检查保证了 C 语言中的一条 break 指令必然位于一个 while/for/switch 语句之内。如果不存在这样的语句，静态检查将报告一个错误。

本章讨论中间表示、静态类型检查和中间代码生成。为了简单起见，我们假设编译器前端的组织如图 6.1 所示，其中解析、静态检查和中间代码生成是按顺序完成的；有时它们可以被组合并折叠到解析中。我们将使用“语法分析”和“语法制导翻译”来具体说明检查和翻译。许多翻译方案可以使用“语法制导翻译”中的技术在自下而上或自上而下的解析过程中实现。所有方案都可以通过创建语法树然后遍历树来实现。

The Benefits of A Good IR



基于一个适当定义的中端表示形式，可以把针对源语言 i 的前端和针对目标机器 j 的后端组合起来，构造得到源语言 i 在目标机器 j 上的一个编译器。这种创建编译器组合的方法可以大大减少工作量：只要写出 m 种前端和 n 种后端处理程序，就可以得到 $m \times n$ 种编译程序。

在编译器的分析-综合模型中，前端分析源程序并创建中间表示，后端从中生成目标代码。理想情况下，源语言的详细信息限制在前端，目标机器的详细信息限制在后端。

通过适当定义的中端表示，可以通过将语言 i 的前端与机器 j 的后端相结合来构建语言 i 和机器 j 的编译器。这种创建编译器套件的方法可以节省大量工作：只需编写 m 个前端和 n 个后端即可构建 $m \times n$ 个编译器

More on IR



Figure 6.2: A compiler might use a sequence of intermediate representations

- A compiler may construct a sequence of IR's
 - **High-level IR's** like syntax trees are close to the source language. They depict the structure of the source program and are suitable for tasks like static type checking
 - **Low-level IR's** are close to the target machines and are suitable for machine-dependent tasks like register allocation and instruction selection
 - Three-address code can be high- or low-level, depending on the choice of operators
- **Interesting fact:** C is often used as an intermediate form. The first C++ compiler has a front end that generates C and a C compiler as a backend

本章介绍的方法可以用于多种中间表示，包括抽象语法树和三地址代码。这两种中间表示术语“三地址代码”来自一般形式 $x = y \text{ op } z$ 的指令，具有三个地址：两个用于操作数 y 和 z ，一个用于综合结果 x 。

在将给定源语言的一个程序翻译成特定的目标机器代码的过程中，一个编译器可能构造出一系列中间表示，如图 6-2 所示。高层的表法接近于源语言，而低层的表示接近于目标机器。语法树是高层的中间表示，它刻画了源程序的自然的层次性结构，并且适用于静态类型检查这样的处理。

低层的表示形式适用于机器相关的处理任务，比如寄存器分配、指令选择等。通过选择不同的运算符，三地址代码既可以是高层的表示方式，也可以是低层的表示方式。在 6.2.3 将看到，对表达式而言，语法树和三地址代码只是在表面上有所不同。对于循环语句，语法树表示了语句的各个组成部分，而三地址代码包含标号和跳转指令，用来表示目标语言的控制流。

不同的编译器对中间表示的选择和设计各有不同。中间表示可以是一种真正的语言，也可以由编译器的各个处理阶段共享的多个内部数据结构组成。C 语言是一种程序设计语言。它具有很好的灵活性和通用性，可以很方便地把 C 程序编译成高效的机器代码，并且

有很多 C 的编译器可用，因此 C 语言也常常被用作中间表示。早期的 C++ 编译器的前端生成 C 代码，而把 C 编译器作为其后端。

在将给定源语言的程序翻译成给定目标机器的代码的过程中，编译器可以构造一系列中间表示，如图 6.2 所示。高级表示接近源语言，低级表示接近目标机器。语法树是高级的；它们描述了源程序的自然层次结构，非常适合静态类型检查等任务。

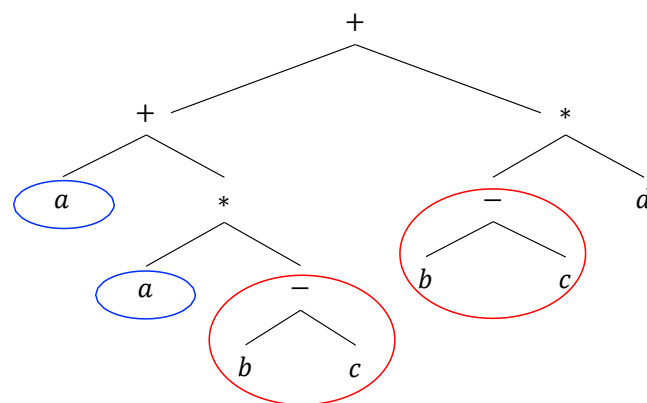
低级表示适用于机器相关的任务，例如寄存器分配和指令选择。三地址代码的范围可以从高级到低级，具体取决于运算符的选择。对于表达式，语法树和三地址代码之间的差异只是表面上的，我们将在第 6.2.3 节中看到。例如，对于循环语句，语法树表示语句的组成部分，而三地址代码包含标签和跳转指令来表示控制流，就像在机器语言中一样。

中间表示的选择或设计因编译器而异。中间表示可以是实际语言，也可以由编译器各阶段共享的内部数据结构组成。C 是一种编程语言，但它经常被用作中间形式，因为它很灵活，可以编译成高效的机器代码，并且它的编译器可以广泛使用。最初的 C++ 编译器由生成 C 的前端组成，将 C 编译器视为后端。

DAG's for Expressions

- In a syntax tree, the tree for a **common subexpression** would be **replicated** as many times as the subexpression appears

▪ Example: $a + a * (b - c) + (b - c) * d$



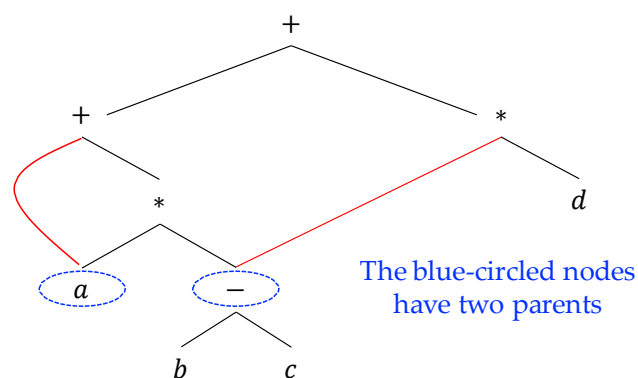
和表达式的语法树类似，一个 DAG 的叶子结点对应于原子运算分量，而内部结点对应于运算符。与语法树不同的是，如果 DAG 中的一个结点 N 表示一个公共子表达式，则

N 可能有多个父结点。在语法树中，公共子表达式每出现一次，代表该公共子表达式的子树就会被复制一次。

DAG's for Expressions Cont.

- A *directed acyclic graph* (DAG, 有向无环图) identifies the common subexpressions and represents expressions succinctly

- Example: $a + a * (b - c) + (b - c) * d$



因此，DAG 不仅更简洁地表示了表达式，而且可以为最终生成表达式的高效代码提供重要的信息。

叶子结点 a 在表达式中出现了两次，因此 a 有两个父结点。值得注意的是，结点 “-” 代表公共子表达式 $b-c$ 的两次出现。该结点同样有两个父结点，表明该子表达式在子表达式 $a*(b-c)$ 和 $(b-c)*d$ 中两次被使用。尽管 b 和 c 在整个表达式中出现了两次，但它们对应的结点只有一个父结点，因为对它们的使用都出现在同样的公共子表达式 $b-c$ 中。

Constructing DAG's

- DAG's can be constructed by the same SDD that constructs syntax trees
- **The difference:** When constructing DAG's, a new node is created if and only if there is no existing identical node

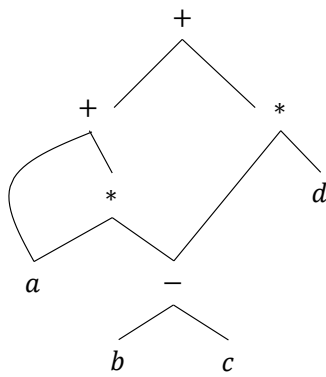
PRODUCTION	SEMANTIC RULES
1) $E \rightarrow E_1 + T$	$E.node = \text{new Node}('+', E_1.node, T.node)$
2) $E \rightarrow E_1 - T$	$E.node = \text{new Node}('-', E_1.node, T.node)$
3) $E \rightarrow T$	$E.node = T.node$
4) $T \rightarrow (E)$	$T.node = E.node$
5) $T \rightarrow \text{id}$	$T.node = \text{new Leaf}(\text{id}, \text{id.entry})$
6) $T \rightarrow \text{num}$	$T.node = \text{new Leaf}(\text{num}, \text{num.val})$

Reuse existing nodes

图 6-4 给出的 SDD（语法制导定义）既可以用来构造语法树，也可以用来构造 DAG。它在例 5.11 中曾用于构造语法树。在那里，函数 Leaf 和 Node 每次被调用都会构造出一个新结点。要构造得到 DAG, 这些函数就要在每次构造新结点之前首先检查是否已存在这样的结点。如果存在一个已被创建的结点，就返回这个已有的结点。例如，在构造一个新结点 Node (op, left, right) 之前，我们首先检查是否已存在一个结点，该结点的标号为 op, 且其两个子结点为 left 和 right。如果存在这样的结点，Node 函数返回这个已存在的结点，否则它创建一个新结点。

Constructing DAG's Cont.

- The construction steps



- 1) $p_1 = \text{Leaf}(\text{id}, \text{entry-a})$
- 2) $p_2 = \text{Leaf}(\text{id}, \text{entry-a}) = p_1$
- 3) $p_3 = \text{Leaf}(\text{id}, \text{entry-b})$
- 4) $p_4 = \text{Leaf}(\text{id}, \text{entry-c})$
- 5) $p_5 = \text{Node}('-', p_3, p_4)$
- 6) $p_6 = \text{Node}('*', p_1, p_5)$
- 7) $p_7 = \text{Node}('+', p_1, p_6)$
- 8) $p_8 = \text{Leaf}(\text{id}, \text{entry-b}) = p_3$
- 9) $p_9 = \text{Leaf}(\text{id}, \text{entry-c}) = p_4$
- 10) $p_{10} = \text{Node}('-', p_3, p_4) = p_5$
- 11) $p_{11} = \text{Leaf}(\text{id}, \text{entry-d})$
- 12) $p_{12} = \text{Node}('*', p_5, p_{11})$
- 13) $p_{13} = \text{Node}('+', p_7, p_{12})$

图 6.5 所示的步骤序列构建了图 6.3 中的 DAG，前提是 Node 和 Leaf 返回现有的节点（如果可能），如下所示上面讨论过。

我们假设 entry-a 指向 a 的符号表条目，对于其他标识符也是如此。当在步骤 2 中重复调用 Leaf(id;entry-a)时，返回先前调用创建的 node，因此 $p_2 = p_1$ 。类似地，步骤 8 和 9 返回的 nodes 与步骤 3 和 4 返回的相同（即 $p_8 = p_3$ 和 $p_9 = p_4$ ）。因此，第 10 步返回的节点必须与第 5 步返回的节点相同；即 $p_{10} = p_5$

Three-Address Code (三地址代码)

- In three-address code, there is **at most one operator** on the right side of an instruction
 - Instructions are often in the form $x = y \text{ op } z$
- **Operands** (or addresses) can be:
 - **Names** in the source programs
 - **Constants**: a compiler must deal with many types of constants
 - **Temporary names** generated by a compiler

在三地址代码中，一条指令的右侧最多有一个运算符；也就是说，不允许使用组合算术表达式。因此，像 $x+y*z$ 这样的源语言表达式可能会被翻译成三地址指令序列：

其中 $t1$ 和 $t2$ 是编译器生成的临时名称。多运算符算术表达式和嵌套控制流语句的这种分解使得三地址代码成为目标代码生成和优化的理想选择，如第 8 章和第 9 章所述。中间值名称的使用通过程序计算可以轻松地重新排列三地址代码。

地址可以是以下之一：

- 名称：为了方便起见，我们允许源程序名称作为三地址代码中的地址出现。

在实现中，源名称由指向其符号表条目的指针替换，其中保留了该名称的所有信息。

- 常量：实际上，编译器必须处理许多不同类型的常量和变量。第 6.5.2 节考虑了表达式内的类型转换。

- 编译器生成的临时文件：每次需要临时文件时创建一个不同的名称非常有用，特别是在优化编译器时。当寄存器分配给变量时，如果可能的话，可以组合这些临时变量。

Instructions (1)

1. **Assignment instructions:**
 - $x = y \text{ op } z$, where op is a binary arithmetic/logical operation
 - $x = \text{op } y$, where op is a unary operation
2. **Copy instructions:** $x = y$
3. **Unconditional jump instructions:** $\text{goto } L$, where L is a label of the jump target
4. **Conditional jump instructions:**
 - $\text{if } x \text{ goto } L$
 - $\text{if Flase } x \text{ goto } L$
 - $\text{if } x \text{ relop } y \text{ goto } L$

现在我们考虑本书其余部分中使用的常见三地址指令。符号标签将由改变控制流的指令使用。符号标签表示指令序列中三地址指令的索引。实际索引可以通过单独传递或通过“回填”来替换标签，第 6.7 节中讨论了这一点。以下是常见三地址指令形式的列表：

1 赋值指令：

$x = y \text{ op } z$ ，其中 op 是二进制算术/逻辑运算， x 、 y 和 z 是地址。

$x = \text{op } y$ ，其中 op 是一元运算。基本的一元运算包括一元减、逻辑非和转换运算符，例如，将整数转换为浮点数。

2 复制指令： $x = y$ ，其中 x 被赋予 y 的值

3 无条件跳转指令： $\text{goto } L$ ，其中 L 是跳转目标的标记，带有标签 L 的三地址指令是下一个要执行的指令。

4 有条件跳转指令：

$\text{if } x \text{ goto } L$

$\text{if Flase } x \text{ goto } L$

$\text{if } x \text{ relop } y \text{ goto } L$

if x goto L 和 ifFalse x goto L 形式的条件跳转。如果 x 为 true 和 false, 这些指令分别执行接下来带有标号 L 的指令。否则, 照常执行接下来的三地址指令

条件跳转, 例如 if x relop y goto L, 它将关系运算符 (<、==、>= 等) 应用于 x 和 y, 如果 x 与 relop 与 y 存在关系, 则接下来执行标号为 L 的指令。如果不是, 则依次执行 if x relop y goto L 后面的三地址指令

Instructions (2)

5. Procedural calls and returns

- param x_1
- ...
- param x_n
- call p, n (procedure call)
- $y = \text{call } p, n$ (function call)
- return y

6. Indexed copy instructions: $x = y[i]$ $x[i] = y$

- Here, $y[i]$ means the value in the location i memory units beyond location y

7. Address and pointer assignment instructions:

- $x = \&y$ $x = *y$ $*x = y$

5 程序调用和返回:

过程调用和返回是使用以下指令实现的: param x 为参数; call p, n 和 y = call p, n 分别用于过程和函数调用; return y, 其中 y 表示返回值, 是可选的。这些三地址指令的常见用法见下面的三地址指令序列

它是 p($x_1 ; x_2 ; \dots ; x_n$) 调用的一部分。“call p, n” 中的 n 是实际参数的数量。这个 n 并不是多余的, 因为调用可以嵌套。也就是说, 前面的一些 param 语句可能是 p 返回之后才执行的某个函数调用的参数, 而 p 的返回值又成为这个后续函数调用的另一个参数

6 带有下标的复制指令:

$x = y[i]$ 这里, $y[i]$ 表示位置 i 存储单元中超出位置 y 的值。

$x[i] = y$ 这里, $y[i]$ 表示位置 i 存储单元中超出位置 y 的值。

7 地址和指针赋值指令：

$x = \&y$, $x = *y$, $*x = y$

$x = \&y$ 、 $x = *y$ 和 $*x = y$ 形式的地址和指针赋值。指令 $x = \&y$ 将 x 的右值设置为 y 的位置（左值）。这个 y 通常 是一个名称，也可能是一个临时变量，它表示一个诸如 `Array` 这样具有左值的表达式。 x 是指针名称或临时指针。在指令 $x = *y$ 中， y 可能是一个指针或一个临时变量，其右值是一个内存位置。这个指令使得 x 的右值等于存储在该位置中的值。最后，指令 $*x = y$ 将 y 的右值 x 赋给由 x 指向的目标的右值 。

Example

- Source code: `do i = i + 1; while (a[i] < v);`

```
L:  t1 = i + 1  
    i = t1  
    t2 = i * 8  
    t3 = a [ t2 ]  
    if t3 < v goto L
```

(a) Symbolic labels.

```
100: t1 = i + 1  
101: i = t1  
102: t2 = i * 8  
103: t3 = a [ t2 ]  
104: if t3 < v goto 100
```

(b) Position numbers.

Assuming each array element takes 8 units of space

该语句的两种可能的翻译如图 6.9 所示。图 6.9(a) 中的翻译使用了附加到第一条指令的符号标签 `L`。(b) 中的翻译显示了每一条指令的位置编号，图中选择以 100 开始。在两个翻译中，最后一条指令都是目标为第一条指令的条件跳转指令。乘法运算 $i*8$ 适用于每个元素占用 8 个存储单元的数组。

选择使用哪些运算符是中间表示形式设计的一个重要问题。显然，这个运算符集合中的运算符要足够丰富，以便实现源语言中的所有运算。接近机器指令的运算符可以使在目标机器上实现中间表示形式更加容易。然而，如果前端必须为某些源语言运算生成很长的

指令序列，那么优化器和代码生成器就需要花费更多的时间去重新发现程序的结构，然后才能为这些运算生成高质量的目标代码。

Representation of Instructions

- The description of three-address instructions specifies the **components of each type of instruction**, but does not specify the representation of instructions
- In a compiler, these instructions can be implemented as **objects/records** with fields for the operator and the operands
- Three typical representations:
 - **Quadruples** (四元式表示方法)
 - **Triples** (三元式表示方法)
 - **Indirect triples** (间接三元式表示方法)

三地址指令的描述指定了每种指令的组成部分，但没有指定这些指令在数据结构中的表示。

在编译器中，这些指令可以实现为对象或带有运算符和操作数字段的记录。

三个这样的表示称为“四元组”、“三元组”和“间接三元组”。

四元式表示方法

三元式表示方法

间接三元式表示方法

Quadruples (四元式)

- A *quadruple* (or “*quad*”) has four fields
 - General form: *op arg₁ arg₂ result*
 - *op* contains an *internal code* for the operator
 - *arg₁, arg₂, result* are *addresses* (operands)
 - Example: $x = y + z \rightarrow + \quad y \quad z \quad x$
- **Unary operators** like $x = \text{minus } y$ or $x = y$ do not use *arg₂*
- **param operators** use neither *arg₂* nor *result*
- **Conditional/unconditional jumps** put the target label in *result*

一个四元组（或 “四元组”）有四个字段

一般形式： *op arg1 arg2 result*

字段 *op* 包含一个运算符的内部代码

arg1、*arg2*、*result* 是地址（操作数）

例如，三地址指令 $x = y + z$ 通过将 $+$ 放入 *op*、将 *y* 放入 *arg1*、将 *z* 放入 *arg2* 以及将 *x* 放入 *result* 来表示。

以下是该规则的一些例外情况

一元运算符（如 $x = -y$ 或 $x = y$ ）不使用 *arg2*

参数运算符既不使用 *arg2* 也不使用 *result*

条件/非条件跳转将目标标签放在 *result* 中

Quadruples Example

- Assignment statement: $a = b * -c + b * -c$

```
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a = t5
```

(a) Three-address code

	op	arg ₁	arg ₂	result
0	minus	c		t ₁
1	*	b	t ₁	t ₂
2	minus	c		t ₃
3	*	b	t ₃	t ₄
4	+	t ₂	t ₄	t ₅
5	=	t ₅		a
		...		

(b) Quadruples

Temporaries

The result field is used primarily for temporary names
Temporary names waste space (symbol table entries)

赋值的三地址代码 $a = b * -c + b * -c$ 的三地址代码如图 6.10(a) 所示。这里我们使用特殊的 minus 运算符来表示“-c”中的单目减运算符“-”，以区别于“b-c”中的双目减运算符“-”。请注意，单目减的三地址语句中只有两个地址，复制语句 $a = t_5$ 也是如此。

为了便于阅读，我们在 arg₁、arg₂ 字段中使用 a、b 和 c 等实际标识符，结果如图 6.10(b) 所示，而不是指向其符号表条目的指针。临时名称可以像程序员定义的名称一样输入到符号表中，也可以实现为 Temp 类的对象，这个 temp 类有自己的方法。

Triples (三元式)

- A *triple* has only three fields: *op*, *arg₁*, *arg₂*
- We refer to the result of an operation *x op y* by its position without generating temporary names (an optimization over quadruples)

$t_1 = \text{minus } c$ $t_2 = b * t_1$ $t_3 = \text{minus } c$ $t_4 = b * t_3$ $t_5 = t_2 + t_4$ $a = t_5$	<table><tr><th></th><th>op</th><th>arg₁</th><th>arg₂</th><th>result</th></tr><tr><td>0</td><td>minus</td><td>c</td><td></td><td>t₁</td></tr><tr><td>1</td><td>*</td><td>b</td><td>t₁</td><td>t₂</td></tr><tr><td>2</td><td>minus</td><td>c</td><td></td><td>t₃</td></tr><tr><td>3</td><td>*</td><td>b</td><td>t₃</td><td>t₄</td></tr><tr><td>4</td><td>+</td><td>t₂</td><td>t₄</td><td>t₅</td></tr><tr><td>5</td><td>=</td><td>t₅</td><td></td><td>a</td></tr><tr><td colspan="5">...</td></tr></table>		op	arg ₁	arg ₂	result	0	minus	c		t ₁	1	*	b	t ₁	t ₂	2	minus	c		t ₃	3	*	b	t ₃	t ₄	4	+	t ₂	t ₄	t ₅	5	=	t ₅		a	...					<table><tr><th></th><th>op</th><th>arg₁</th><th>arg₂</th></tr><tr><td>0</td><td>minus</td><td>c</td><td></td></tr><tr><td>1</td><td>*</td><td>b</td><td>(0)</td></tr><tr><td>2</td><td>minus</td><td>c</td><td></td></tr><tr><td>3</td><td>*</td><td>b</td><td>(2)</td></tr><tr><td>4</td><td>+</td><td>(1)</td><td>(3)</td></tr><tr><td>5</td><td>=</td><td>a</td><td>(4)</td></tr><tr><td colspan="4">...</td></tr></table>		op	arg ₁	arg ₂	0	minus	c		1	*	b	(0)	2	minus	c		3	*	b	(2)	4	+	(1)	(3)	5	=	a	(4)	...			
	op	arg ₁	arg ₂	result																																																																						
0	minus	c		t ₁																																																																						
1	*	b	t ₁	t ₂																																																																						
2	minus	c		t ₃																																																																						
3	*	b	t ₃	t ₄																																																																						
4	+	t ₂	t ₄	t ₅																																																																						
5	=	t ₅		a																																																																						
...																																																																										
	op	arg ₁	arg ₂																																																																							
0	minus	c																																																																								
1	*	b	(0)																																																																							
2	minus	c																																																																								
3	*	b	(2)																																																																							
4	+	(1)	(3)																																																																							
5	=	a	(4)																																																																							
...																																																																										
Three-address code	Quadruples	Triples																																																																								

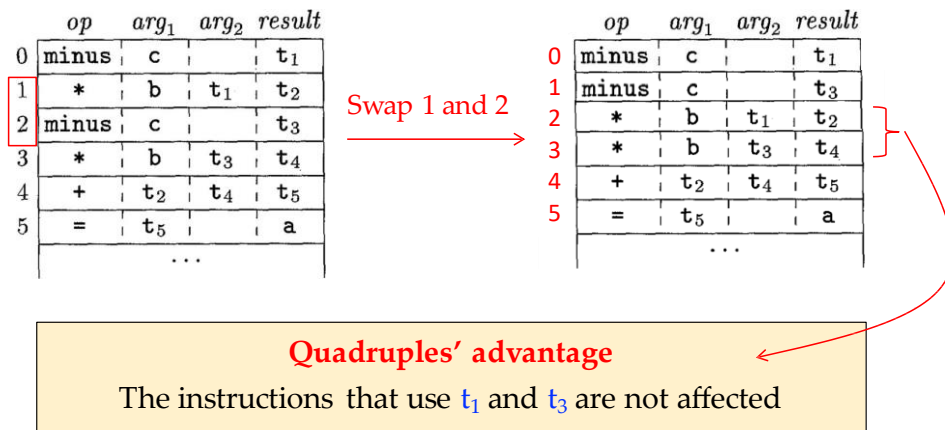
一个三元式(triple)只有三个字段，我们分别称之为 *op*、*arg₁* 和 *arg₂*。请注意，图 6-10b 中的 *result* 字段主要被用于临时变量名。

使用三元式时，我们将用运算 *X op J* 的位置来表示它的结果，而不是用一个显式的临时名字表示。

例如，在三元式表示中将直接用位置(0)，而不是像图 6-10b 中那样用临时名字 *t₁* 来表示对相应运算结果的引用。带有括号的数字表示指向相应三元式结构的指针。在 6.1.2 节中，位置或指向位置的编码被称为值编码。

Problem of Triples

- In optimizing compilers, instructions are often moved around

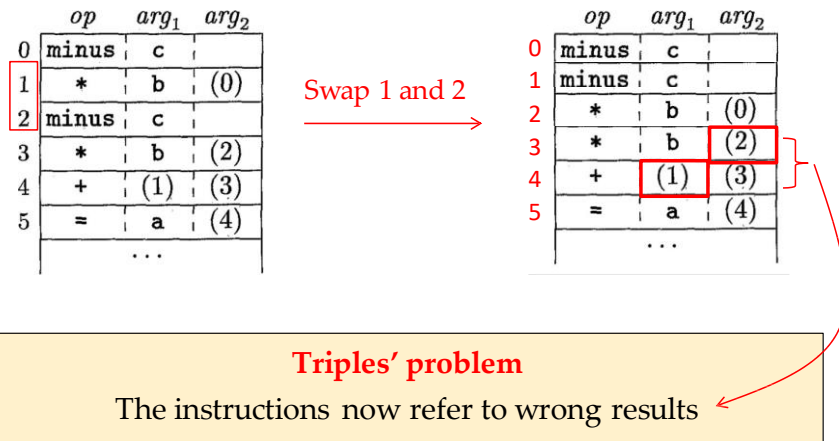


在优化编译器中，由于指令的位置常常会发生变化，四元式相对于三元式的优势就体现出来了。

使用四元式时，如果我们移动了一个计算临时变量 t 的指令，那些使用 t 的指令不需要做任何改变。而使用三元式时，对于运算结果的引用是通过位置完成的，因此如果改变一条指令的位置，则引用该指令的结果的所有指令都要做相应的修改。

Problem of Triples Cont.

- In optimizing compilers, instructions are often moved around

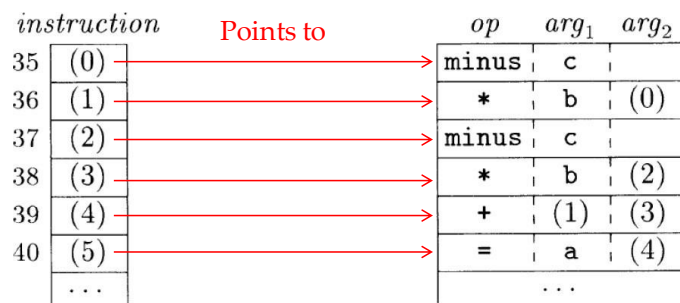


三地址的问题:

说明现在提到了错误的结果, 位置变了

Indirect Triples (间接三元式)

- Indirect triples* consist a list of **pointers** to triples (the *instruction* array below)

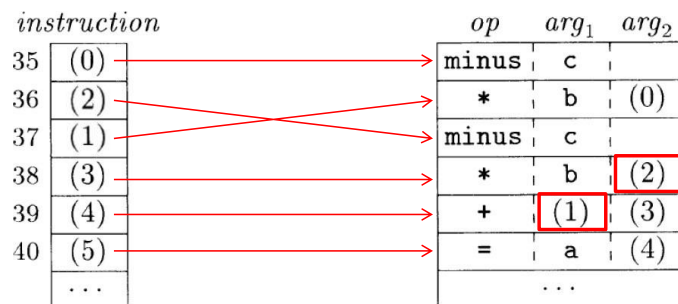


间接三元组由一个指向三元组的指针列表（下面的指令数组）

间接三元式(indirect triple)包含了一个指向三元式的指针的列表，而不是列出三元式序列本身。例如，我们可以使用数组 *instruction* 按照适当的顺序列出指向三元式的指针。这样，图 6-11b 中的三元式序列就可以表示成为图 6-12 所示的形式。

Indirect Triples (间接三元式)

- An optimization can move an instruction by reordering the *instruction* list (without modifying the triples)



Pointers!

After swapping, the 1st and 2nd instructions in the list still point to the right ones

优化可以通过重新排序指令列表来移动指令（无需修改三元组）

交换后，列表中的第一条和第二条指令仍然指向正确的指令

Static Single-Assignment Form (1)

- **Static single-assignment** form (SSA, 静态单赋值形式) is an IR that facilitates certain code optimizations
- In SSA, each name receives **a single assignment**

```
p = a + b
q = p - c
p = q * d
p = e - p
q = p + q
```

```
p1 = a + b
q1 = p1 - c
p2 = q1 * d
p3 = e - p2
q2 = p3 + q1
```

(a) Three-address code. (b) Static single-assignment form.

静态单赋值形式静态单赋值形式（SSA，Static Single-Assignment FormStatic single-assignment form）是一种 IR，有助于某些代码优化。

在 SSA 中，每个名称只接受一次赋值。

SSA 和三地址代码的区别主要体现在两个方面。首先，SSA 中的所有赋值都是针对具有不同名字的变量的，这也是“静态单赋值”这一名字的由来。

图 6-13 给出了分别以三地址代码形式和静态单赋值形式表示的中间程序。注意，SSA 表示中对变量 p 和 q 的每次定值都以不同的下标加以区分。

Static Single-Assignment Form (2)

- The same variable may be defined in two different control-flow paths

```
if ( flag ) x = -1; else x = 1;  
y = x * a;
```



Which name should we use in $y = x * a$?

同一变量可在两个不同的控制流路径中定义

在一个程序中，同一个变量可能在两个不同的控制流路径中被定值。例如，下列源程序

中， x 在两个不同的控制流路径中被定值。如果我们对条件语句

的真分支和假分支中的 x 使用不同的变量名，那么我们应该在赋值运算 $y=x*a$ 中使用哪个名字？

Static Single-Assignment Form (3)

- The same variable may be defined in two different control-flow paths

```
if ( flag ) x = -1; else x = 1;  
y = x * a;
```

- SSA uses a notational convention called ϕ -function to combine the two definitions of x

```
if ( flag )  $x_1$  = -1; else  $x_2$  = 1;  
 $x_3$  =  $\phi(x_1, x_2)$ ; //  $x_1$  if control flow passes through the true part; otherwise  $x_2$   
y =  $x_3$  * a;
```

同一变量可在两个不同的控制流路径中定义:SSA 使用一种称为 ϕ 函数的符号惯例来合并 x 的两个定义

这也是 SSA 的第二个特别之处。SSA 使用一种被称为 ϕ 函数的表示规则将 x 的两处定值合并起来:

如果控制流经过这个条件语句的真分支, $\phi(x_1, x_2)$ 的值为 x_1 ; 否则, 如果控制流经过假分支, ϕ 函数的值为 x_2 。也就是说, 根据到达包含 ϕ 函数的赋值语句的不同控制流路径, ϕ 函数返回不同的参数值。

Types and Type Checking

- *Data type* or simply *type* tells a compiler or interpreter how the programmers intend to use the data
- The usefulness of type information
 - Find faults in the source code
 - Determine the storage needed for a name at runtime
 - Calculate the address of an array element
 - Insert type conversions
 - Choose the right version of an arithmetic operator (e.g., `fadd`, `iadd`)
- *Type checking* (类型检查) uses logical rules to make sure that the types of the operands match the type expected by an operator

数据类型或简单的类型告诉编译器或解释器程序员打算如何使用数据

类型信息的作用

查找源代码中的错误

在运行时确定名称所需的存储空间

计算数组元素的地址

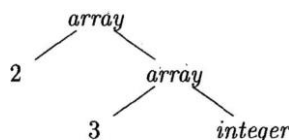
插入类型转换

选择算术运算符的正确版本（如 `fadd`、`iadd`）

类型检查(类型检查) 使用逻辑规则确保操作数的类型与运算符所期望的类型匹配。更明确地讲，类型检查保证运算分量的类型和运算符的预期类型相匹配。例如，Java 要求 `&` 运算符的两个运算分量必须是 `boolean` 型。如果满足这个条件，结果也具有 `boolean` 类型。

Type Expressions (类型表达式)

- **Types have structure**, which can be represented by *type expressions*
 - A type expression is either a basic type, or
 - Formed by applying a *type constructor* (类型构造算子) to a type expression
- **`array(2, array(3, integer))`** is the type expression for `int[2][3]`
 - **`array`** is a type constructor with two arguments: a number, a type expression



类型具有结构，可以用类型表达式来表示：

类型表达式可能是基本类型，

也可能通过把被称为类型构造算子的运算符作用于类型表达式而得到

数组类型 `int[2][3]` 表示 “由两个数组组成的数组，其中的每个数组各包含 3 个整数”。

它的类型表达式可以写成 `array (2, array (3, integer))`。该类型可以用如图 6-14 所示的树来描述。`array` 运算符有两个参数：一个数字和一个类型。

The Definition of Type Expression

- A basic type is a type expression
 - *boolean, char, integer, float, and void, ...*
- A type name (e.g., name of a class) is a type expression
- A type expression can be formed
 - By applying the *array* type constructor to a number and a type expression
 - By applying the *record* type constructor to the field names and their types
 - By applying the $\rightarrow$ type constructor for function types
- If *s* and *t* are type expressions, then their Cartesian product $s \times t$ is a type expression (e.g., represent a tuple of types such as function parameters)

* Type expressions may contain type variables (e.g., those generated by compilers) whose values are type expressions

基本类型是一种类型表达式

类型名称（如类名）是一个类型表达式

类型表达式可以通过以下方式形成

对数字和类型表达式应用 *array* 类型构造函数

对字段名及其类型应用 *record* 类型构造函数

对函数类型应用 $\rightarrow$ 类型构造函数

如果 *s* 和 *t* 是类型表达式，那么它们的笛卡尔乘积 $s \times t$ 就是一个类型表达式（例如，表示函数参数等类型的元组）。

* 类型表达式可能包含类型变量（例如由编译器生成的变量），其值为类型表达式

Type Equivalence

- Type checking rules usually have the following form

If two type expressions are equivalent
then return a given type
else return **type_error**

Code under analysis:
`a + b`

- The key is to define when two type expressions are equivalent
 - **The main difficulty** arises from the fact that most modern languages allow the naming of user-defined types
 - In C/C++, type naming is achieved by the `typedef` statement

类型检查规则通常有以下形式

如果两个类型表达式等价，则返回给定类型，否则返回 `type_error`

关键是定义两个类型表达式何时等价

主要的困难来自于大多数现代语言都允许对用户定义的类型进行命名。

在 C/C++ 中，类型命名是通过 `typedef` 语句实现的

Name Equivalence (名等价)

- Treat named types as basic types; **names in type expressions are not replaced** by the exact type expressions they define
- Two type expressions are name equivalent if and only if **they are identical** (represented by the same syntax tree, with the same labels)

```
typedef struct { int  
    data[100]; int  
    count;  
} Stack;
```

```
typedef struct { int  
    data[100]; int  
    count;  
} Set;
```

Code under analysis:

Stack x, y;

Set r, s;

x = y; ✓

r = s; ✓

x = r; ✗

<http://web.eecs.utk.edu/~bvanderz/teaching/cs365Sp14/notes/types.html>

将命名的类型视为基本类型；类型表达式中的名称不会被其定义的类型表达式所替换
当且仅当两个类型表达式完全相同时，它们的名称才是等价的(由相同的语法树表示，
具有相同的标签)

Structural Equivalence (结构等价)

- For named types, replace the type names by their type expressions and recursively check the substituted trees

```
typedef struct { int  
    data[100]; int  
    count;  
} Stack;
```

```
typedef struct { int  
    data[100]; int  
    count;  
} Set;
```

Code under analysis:

```
Stack x, y;
```

```
Set r, s;
```

```
x = y; ✓
```

```
r = s; ✓
```

```
x = r; ✓
```

对于已命名的类型，用其类型表达式替换类型名，并递归检查替换后的树

Declarations (类型的声明)

- The grammar below deals with basic, array, and record types
 - Nonterminal *D* generates a sequence of declarations
 - T* generates basic, array, or record types, A record type is a sequence of declarations for the fields of the record, surrounded by curly braces
 - B* generates one of the basic types: `int` and `float`
 - C* generates sequences of one or more integers, each surrounded by brackets

$$\begin{aligned} D &\rightarrow T \text{ id } ; D \mid \epsilon \\ T &\rightarrow B C \mid \text{record } \{ D \} \\ B &\rightarrow \text{int} \mid \text{float} \\ C &\rightarrow \epsilon \mid [\text{num}] C \end{aligned}$$

下面的语法涉及基本类型、数组类型和记录类型

非终结符号 D 生成一系列声明。

非终结符号 T 生成基本类型、数组类型或记录类型。

非终结符号 B 生成基本类型 `int` 和 `float` 之一。

非终结符号 C（表示“分量”）产生零个或多个整数，每个整数用方括号括起来。

一个数组类型包含一个由 B 指定的基本类型，后面跟一个由非终结符号 C 指定的数组分。一个记录类型(T 的第二个产生式) 由各个记录字段的声明序列构成，并被花括号括起来。

Storage Layout for Local Names (局部变量的存储布局)

- From the type of a name, we can decide the amount of memory needed for the name at run time
 - The *width (宽度)* of a type: # memory units needed for an object of the type
 - For data of varying lengths, such as strings, or whose size cannot be determined until run time, such as dynamic arrays, we only reserve a fixed amount of memory for a pointer to the data
- For **local names of a function**, we always assign contiguous bytes*
 - For each such name, at compile time, we can compute a **relative address**
 - Type information and relative addresses are stored in **symbol table**

* This follows the principle of proximity and is mainly for performance considerations.

根据名称的类型，我们可以决定运行时名称所需的内存量。运行时名称所需的内存量

类型的宽度（宽度）： 该类型对象所需的 # 内存单位

对于长度不等的的数据（如字符串），或在运行前无法确定大小的数据（如动态数组），我们只为数据指针预留固定的内存量。

对于函数的局部名称，我们总是分配连续的字节*。

对于每个这样的名称，我们可以在编译时计算出一个相对地址

类型信息和相对地址存储在符号表中

* 这遵循就近原则，主要是出于性能考虑。

An SDT for Computing Types and Their Widths

- **Synthesized attributes**: *type, width*
- Global variables *t* and *w* pass type and width information from a *B* node in a parse tree to the node for the production $C \rightarrow \epsilon$
 - In an SDD, *t* and *w* would be *C*'s **inherited attributes** (the SDD is L-attributed)*

$T \rightarrow B$	$\{ t = B.type; w = B.width; \}$
C	$\{ T.type = C.type; T.width = C.width; \}$
$B \rightarrow \text{int}$	$\{ B.type = \text{integer}; B.width = 4; \}$
$B \rightarrow \text{float}$	$\{ B.type = \text{float}; B.width = 8; \}$
$C \rightarrow \epsilon$	$\{ C.type = t; C.width = w; \}$
$C \rightarrow [\text{num}] C_1$	$\{ C.type = \text{array}(\text{num.value}, C_1.type); \\ C.width = \text{num.value} \times C_1.width; \}$

This SDT can be implemented during recursive-descent parsing

合成属性：类型、宽度

全局变量 *t* 和 *w* 将类型和宽度信息从解析树中的 *B* 节点传递到生成 $C \rightarrow \epsilon$ 的节点。

节点传递类型和宽度信息。在 SDD 中，*t* 和 *w* 是 *C* 的继承属性（SDD 是 L 属性）*。（该 SDT 可以在递归下降分析过程中调用）

T 产生式的产生式体包含一个非终结符号 *B*、一个动作和一个非终结符号 *C*，其中 *C* 显示在下一行上。*B* 和 *C* 之间的动作是将 *t* 设置为 *B.type*，并将 *w* 设置为 *B.width*。

如果 $B \rightarrow \text{int}$ ，则 *B.type* 被设置为 *integer*，*B.width* 被设置为 4，即一个整型数的宽度。

类似的，如果 $B \rightarrow \text{float}$ ，则 *B.type* 和 *B.width* 分别被设置为 *float* 和 8，即宽度为一个浮点数的宽度。

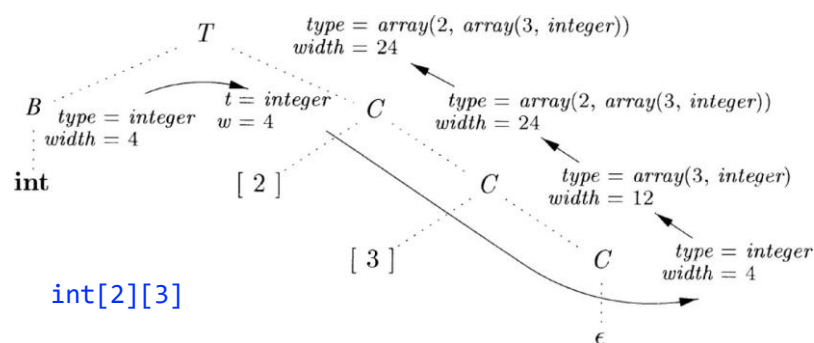
C 的产生式决定了 *T* 生成的是一个基本类型还是一个数组类型。如果 $C \rightarrow \epsilon$ ，则 *t* 变成 *C.type*，且 *w* 变成 *C.width*。

否则，C 就描述了一个数组分量。C → [num]C1 的动作将类型构造算子 array 应用于运算分值 num. value 和 C1. type,构造得到 C. type。例如，应用 array 的结果可能是图 6-14 所示的树形结构。

数组的宽度是将数组元素的个数乘以单个数组元素的宽度而得到。如果连续存放的整数的地址之间的差距为 4,那么一个整数数组的地址计算将包含乘 4 运算。这样的乘法运算为优化提供了机会，因此让前端程序在其输出中明确描述这些运算将有助于优化。在这一章中，我们将忽略其他与机器相关特性，比如数据对象的地址必须和机器字的边界对齐。

Translation Process Example

- Recall the translation during recursive-descent parsing
 - Use the arguments of function A() to pass nonterminal A's **inherited attributes***
 - Evaluate and Return the **synthesized attributes** of A when the A() completes



* In our example, we use global variables *t* and *w*

在递归下降解析过程中调用翻译

使用函数的参数 A()传递非终结符 A' s 继承属性* （在例子中我们使用全局变量 t 和 w)

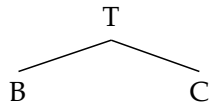
计算并返回的综合属性A 当 A()完成

类型 int[2] [3]的语法分析树用图 6-16 中的虚线描述。图中的实线描述了 type 和 width 是如何从 B 结点开始，通过变量 t 和 w,沿着多个 C 组成的链下传，然后又作为综合属性 type 和 width 沿此链返回的。在访问包含 C 结点的子树之前，变量 t 和 w 被赋予 B. type

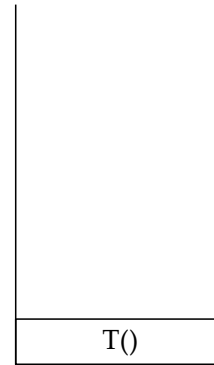
和 B.width 的值。变量 t 和 w 的值在 $C \rightarrow \epsilon$ 对应的结点上使用，然后开始沿着多个 C 结点组成的链向上对综合属性求值。

Translation Process Example

Input string: `int[2][3]`



Step 1: Rewrite T using $T \rightarrow BC$

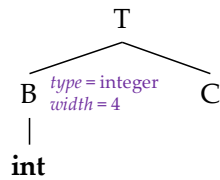


Call stack

步骤 1: 使用 $T \rightarrow BC$ 重写 T

Translation Process Example

Input string: `int[2][3]`



Step 2:

- Rewrite B using $B \rightarrow \mathbf{int}$
- Match input
- Execute semantic action

$B \rightarrow \mathbf{int}$ $\{ B.type = integer; B.width = 4; \}$

B()
T()

Call stack

步骤 2:

使用 $B \rightarrow \mathbf{int}$ 重写 B 。

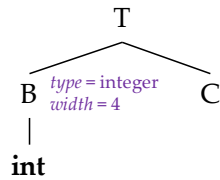
匹配输入

执行语义操作

Translation Process Example

Input string: `int[2][3]`

`t = integer`
`w = 4`

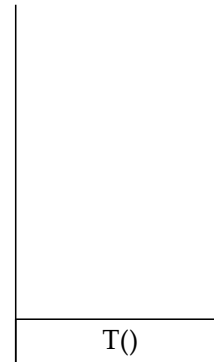


Step 3:

- B() returns
- Execute semantic action

$T \rightarrow B$
 C

$\{ t = B.type; w = B.width; \}$
 $\{ T.type = C.type; T.width = C.width; \}$



Call stack

步骤 3:

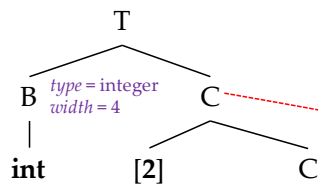
B() 返回

执行语义操作

Translation Process Example

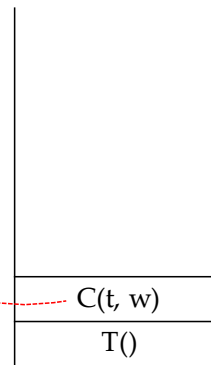
Input string: `int[2][3]`

$t = \text{integer}$
 $w = 4$



Step 4:

- Rewrite C using $C \rightarrow [\text{num}]C$
- Match input



Call stack

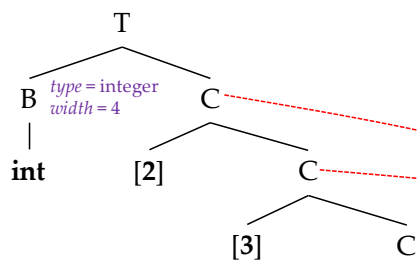
使用 $C \rightarrow [\text{num}]C$ 重写 C

匹配输入

Translation Process Example

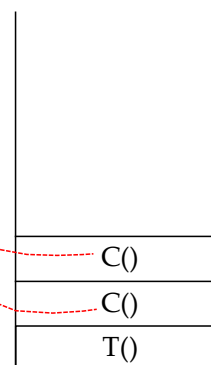
Input string: `int[2][3]`

$t = \text{integer}$
 $w = 4$



Step 5:

- Rewrite C using $C \rightarrow [\text{num}]C$
- Match input



Call stack

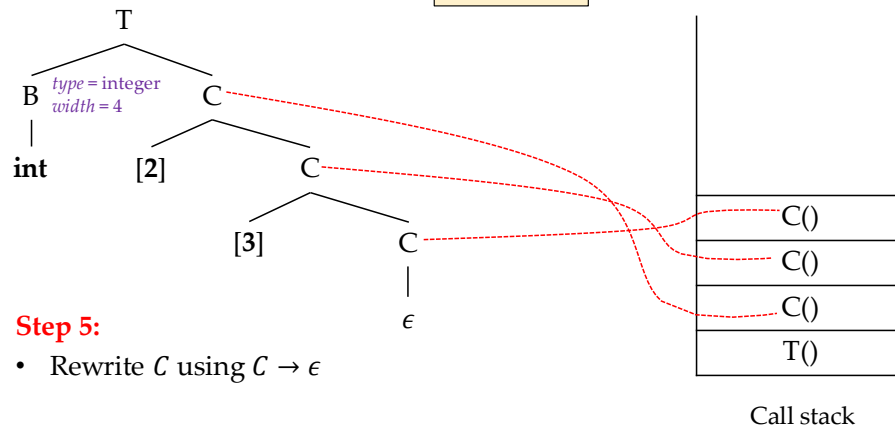
使用 $C \rightarrow [\text{num}]C$ 重写 C

匹配输入

Translation Process Example

Input string: `int[2][3]`

$t = \text{integer}$
 $w = 4$

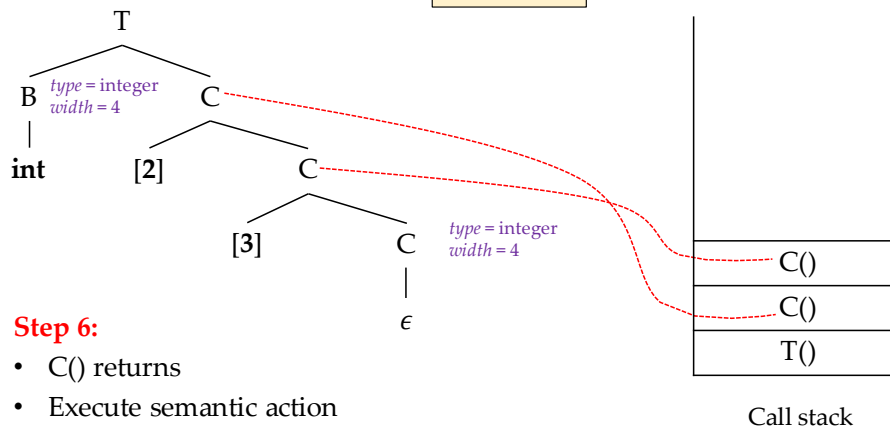


使用 $C \rightarrow \epsilon$ 重写

Translation Process Example

Input string: `int[2][3]`

$t = \text{integer}$
 $w = 4$



$C \rightarrow \epsilon$ $\{ C.type = t; C.width = w; \}$

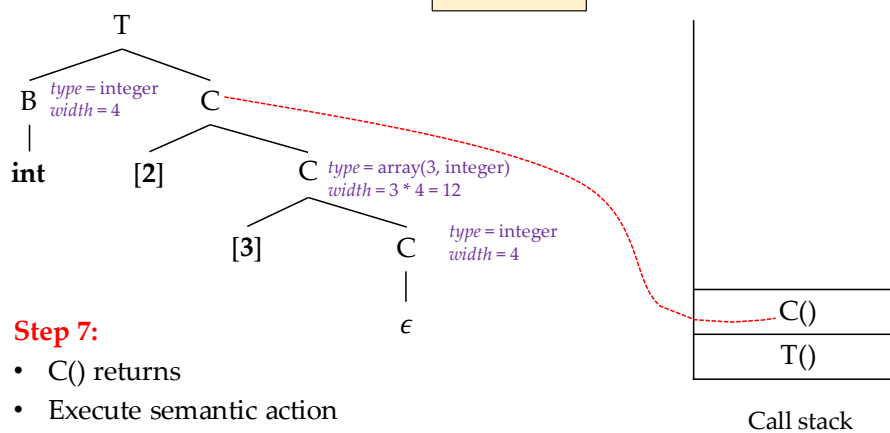
C() 返回

执行语义操作

Translation Process Example

Input string: `int[2][3]`

$t = \text{integer}$
 $w = 4$



$C \rightarrow [\text{num}] C_1$ $\{ C.type = \text{array}(\text{num.value}, C_1.type);$
 $C.width = \text{num.value} \times C_1.width; \}$

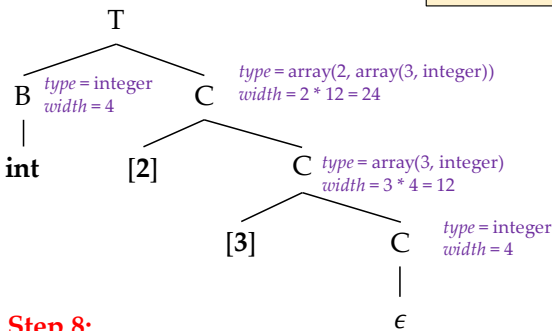
C() 返回

执行语义操作

Translation Process Example

Input string: `int[2][3]`

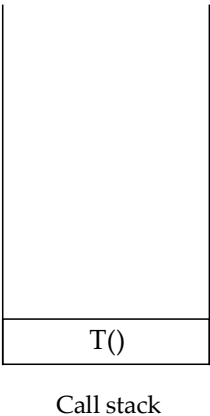
`t = integer`
`w = 4`



Step 8:

- C() returns
- Execute semantic action

$C \rightarrow [\text{num}] C_1$ $\{ \begin{array}{l} C.type = \text{array}(\text{num.value}, C_1.type); \\ C.width = \text{num.value} \times C_1.width; \end{array} \}$



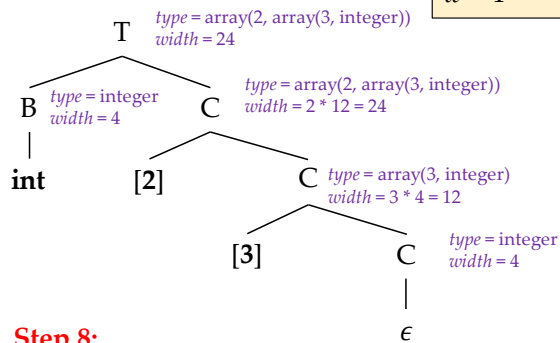
C() 返回

执行语义操作

Translation Process Example

Input string: `int[2][3]`

`t = integer`
`w = 4`



Step 8:

- `T()` returns
- Execute semantic action

$T \rightarrow B$ $\{ t = B.type; w = B.width; \}$
 C $\{ T.type = C.type; T.width = C.width; \}$

Call stack

`T()` 返回

执行语义操作

Sequences of Declarations

- When dealing with a procedure, local variables should be put in a separate symbol table; their declarations can be processed as a group
 - **Name**, **type**, and **relative address** of each variable should be stored
- The translation scheme below handles a sequence of declarations
 - **offset**: the next available relative address; **top**: the current symbol table

$P \rightarrow D \quad \{ offset = 0; \}$
 $D \rightarrow T \text{ id } ; \quad \{ top.put(id.lexeme, T.type, offset);$
 $offset = offset + T.width; \}$
 $D \rightarrow D_1$
 $D \rightarrow \epsilon$

Computing relative addresses of declared names

像 C 和 Java 这样的语言支持将单个过程中的所有声明作为一个组进行处理。这些声明可能分布在一个 Java 过程中，但是仍然能够在分析该过程时处理它们。因此，我们可以使用一个变量，比如 `offset`，来跟踪下一个可用的相对地址。

图 6-17 中的翻译方案处理形如 `T id` 的声明的序列，其中的 `T` 如图 6-15 所示产生一个类型。

在考虑第一个声明之前，`offset` 被设置为 0。每处理一个变量 `x` 时，`x` 被加入符号表，它的相对地

址被设置为 `offset` 的当前值。随后，`x` 的类型的宽度被加到 `offset` 上。

产生式 $D \rightarrow T \text{ id} ; D1$ 中的语义动作首先执行 `top.put(id.lexeme, T.type, offset)`，创建一个符号表条目。这里的 `top` 指向当前的符号表。方法 `top.put` 为 `id.lexeme` 创建一个符号表条目，该条目的数据区中存放了类型 `T.type` 和相对地址 `offset`。

如果我们把第一个产生式写在同一行中：

$$P \rightarrow \{ \text{offset} = 0 ; \} D \quad (6.1)$$

则图 6-17 中对 `offset` 的初始化处理就变得更加容易理解。生成 ϵ 的非终结符号称为标记非终结符号，其作用是重写产生式，使得所有的语义动作都出现在产生式右部的尾端，具体方法见 5.5.4 节。

Fields in Records and Classes*

- Two assumptions:
 - The field names within a record must be distinct
 - The offset for a field name is relative to the data area (数据区) for that record
- For convenience, we use a symbol table for each record type
 - Store both type and relative address of fields
- A record type has the form *record(t)*
 - *record* is the type constructor
 - *t* is a symbol table object, holding info about the fields of this record type

* Self-study materials

图 6-17 中对声明的翻译方案还可以用于处理记录和类中的字段。要把记录类型加入到图 6-15 所示的文法中，只需要加上下面的产生式：

$$T \rightarrow \text{record}\{ 'D' \}$$

这个记录类型中的字段由 D 生成的声明序列描述。图 6-17 中的方法可以用来确定这些字段的类型和相对地址，当然我们需要小心地处理下面两件事：

- 一个记录中各个字段的名称必须是互不相同的。也就是说，在由 D 生成的声明中，同一个名称最多出现一次。

- 字段名的偏移量，或者说相对地址，是相对于该记录的数据区字段而言的。

为了方便起见，记录类型将使用一个专用的符号表，对它们的各个字段的类型和相对地址进行编码。

记录类型形如 $\text{record}(t)$ ，其中 record 是类型构造算子， t 是一个符号表对象，它保存了有关该记录类型的各个字段的信息。

记录类型的形式为 $\text{record}(t)$

record 是类型构造函数

t 是一个符号表对象，包含该记录类型字段的信息

Fields in Records and Classes

$T \rightarrow \text{record}\{ ' \{ ' \{ Env.push(top); top = \text{new Env}(); \}$
$D ' \} ' \{ T.type = \text{record}(top); T.width = offset; \}$
$top = Env.pop(); offset = Stack.pop(); \}$

- The class *Env* implements symbol tables
- *Env.push(top)* and *Stack.push(offset)* save the current symbol table and offset; later, they will be popped to continue with other translation
- The translation scheme can be adapted to deal with classes

为了使翻译方案更加具体，图 6-18 中的动作给出了某个实现的伪代码。

令 Env 类实现符号表。

对 Env.push(top)的调用将 top 所指的当前符号表压入一个栈中。然后，变量 top 被设置为指向一个新的符号表。类似的，offset 被推入名为 Stack 的栈中，offset 变量被重置为 0。

在 D 中的声明被翻译之后，符号表 top 保存了这个记录中所有字段的类型和相对地址。而且，offset 还给出了存放所有字段所需的存储空间。第二个动作将 T.type 设为 record(top)，并将 T.width 设为 offset。然后，变量 top 和 offset 将被恢复为原先被压入栈中的值，以完成这个记录类型的翻译。

有关记录类型存储方式的讨论还可以被推广到类，因为我们无需为类中的方法保留存储空间。

Expressions and Arrays

- An expression with more than one operator: $a + b * c$
 - Translate into multiple instructions with at most one operator per instruction

$$a + b * c \quad \longrightarrow \quad \begin{array}{l} t_1 = b * c \\ t_2 = a + t_1 \end{array}$$

- An array reference $A[i][j]$ will expand into a sequence of three-address instructions that calculate an **address** for the reference

一个带有多个运算符的表达式（比如 $a+b*c$ ）将被翻译成为每条指令最多包含一个运算符的指令序列。

一个数组引用 $A[i][j]$ 将被扩展成一个计算该引用的地址的三地址指令序列。我们将在 6.5 节中考虑表达式的类型检查，并在 6.6 节中介绍如何使用布尔表达式来处理程序的控制流。

SDD for Expressions (1)

PRODUCTION	SEMANTIC RULES
$S \rightarrow \text{id} = E ;$	$S.code = E.code \parallel$ $gen(top.get(id.lexeme) \text{'=' } E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \text{new Temp}()$ $E.code = E_1.code \parallel E_2.code \parallel$ $gen(E.addr \text{'=' } E_1.addr \text{'+' } E_2.addr)$
$ - E_1$	$E.addr = \text{new Temp}()$ $E.code = E_1.code \parallel$ $gen(E.addr \text{'=' 'minus' } E_1.addr)$
$ (E_1)$	$E.addr = E_1.addr$ $E.code = E_1.code$
$ \text{id}$	$E.addr = top.get(id.lexeme)$ $E.code = \text{''}$

S.code and *E.code* denote three-address code

E.addr denotes the address that will hold the value of *E*

top denotes the current symbol table; *get* returns the address of *id* (a variable)

gen generates three-address instructions

All attributes are synthesized. This **S-attributed SDD** can be implemented during bottom-up parsing.

图 6-19 中的语法制导定义使用 *S* 的属性 *code* 以及表达式 *E* 的属性 *addr* 和 *code*, 为一个赋值语句 *S* 生成三地址代码。

属性 *S.code* 和 *E.code* 分别表示 *S* 和 *E* 对应的三地址代码。
属性 *E.addr* 则表示存放 *E* 的值的地址。回忆一下 6.2.1 节，一个地址可以是变量名字、常量或编译器产生的临时量。

SDD for Expressions (2)

PRODUCTION	SEMANTIC RULES
$S \rightarrow \text{id} = E ;$	$S.code = E.code \parallel$ $gen(top.get(\text{id.lexeme}) \neq E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \text{new Temp}()$ $E.code = E_1.code \parallel E_2.code \parallel$ $gen(E.addr \neq E_1.addr \neq E_2.addr)$
$ - E_1$	$E.addr = \text{new Temp}()$ → Temporary name generated by compiler $E.code = E_1.code \parallel$ $gen(E.addr \neq \text{'minus'} E_1.addr)$
$ (E_1)$	$E.addr = E_1.addr$ $E.code = E_1.code$
$ \text{id}$	$E.addr = top.get(\text{id.lexeme})$ → Check the symbol-table entry for id and save its address in E.addr $E.code = ''$

考虑图 6-19 中语法制导定义的最后产生式 $E \rightarrow \text{id}$ 。若表达式只是一个标识符，比如说 x ，那么 x 本身就保存了这个表达式的值。这个产生式对应的语义规则把 $E.addr$ 定义为指向该 id 的实例对应的符号表条目的指针。

令 top 表示当前的符号表。当函数 $top.get$ 被应用于 id 的这个实例的字符串表示 id.lexeme 时，它返回对应的符号表条目。 $E.code$ 被设置为空串。

当规则为 $E \rightarrow (E_1)$ 时，对 E 的翻译与对子表达式 E_1 的翻译相同。因此， $E.addr$ 等于 $E_1.addr$ ， $E.code$ 等于 $E_1.code$ 。

图 6-19 中的运算符 $+$ 和单目 $-$ 是典型语言中的运算符的代表。 $E \rightarrow E_1 + E_2$ 的语义规则生成了根据 E_1 和 E_2 的值计算 E 的值的代码。计算得到的值存放在新生成的临时变量中。如果 E_1 的值计算后被放入 $E_1.addr$ ， E_2 的值被放到 $E_2.addr$ 中，那么 $E_1 + E_2$ 就可以被翻译为 $t = E_1.addr + E_2.addr$ ，其中 t 是一个新的临时变量。 $E.addr$ 被设为 t 。连续执行 $\text{new Temp}()$ 会产生一系列互不相同的临时变量 $t_1, t_2, \dots$ 。

SDD for Expressions (3)

PRODUCTION	SEMANTIC RULES
$S \rightarrow \text{id} = E ;$	$S.code = E.code \parallel$ $gen(top.get(id.lexeme) \neq E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = new Temp()$ $E.code = E_1.code \parallel E_2.code \parallel$ $gen(E.addr \neq E_1.addr \neq E_2.addr)$
$ - E_1$	$E.addr = new Temp()$ $E.code = E_1.code \parallel$ $gen(E.addr \neq \text{'minus'} E_1.addr)$
$ (E_1)$	$E.addr = E_1.addr$ $E.code = E_1.code$
$ \text{id}$	$E.addr = top.get(id.lexeme)$ $E.code = \text{''}$

Concatenate instructions

为方便起见，我们使用记号 $gen(x \neq y \neq z)$ 来表示三地址指令 $x = y + z$ 。当被传递给 gen 时，变量 x 、 y 、 z 的位置上出现的表达式将首先被求值，而像 $\neq$ 这样的引号内的字符串则按照字面值传递。其他的三地址指令的生成方法类似，也是将 gen 作用于表达式和字符串的组合。

当我们翻译产生式 $E \rightarrow E_1 + E_2$ 时，图 6-19 中的语义规则首先将 $E_1.code$ 和 $E_2.code$ 连接起来，然后再加上一条将 E_1 和 E_2 的值相加的指令，从而生成 $E.code$ 。新增加的这条指令将求和的结果放入一个为 E 生成的临时变量中，用 $E.addr$ 表示。

产生式 $E \rightarrow -E_1$ 的翻译过程与此类似。这个规则首先为 E 创建一个新的临时变量，并生成一条指令来执行单目 $-$ 运算。

最终，产生式 $S \rightarrow \text{id} = E;$ 所生成的指令将表达式 E 的值赋给标识符 id 。和规则 $E \rightarrow \text{id}$ 中一样，这个产生式的语义规则使用函数 $top.get$ 来确定 id 所代表的标识符的地址。 $S.code$ 包含的指令首先计算 E 的值并将其保存到由 $E.addr$ 指定的地址中，然后再将这个值赋给这个 id 实例的地址 $top.get(id.lexeme)$ 。

Inefficiency in the SDD

PRODUCTION	SEMANTIC RULES
$S \rightarrow \text{id} = E ;$	$S.code = E.code$ $gen(top.get(id.lexeme) \neq E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \text{new Temp}()$ $E.code = E_1.code E_2.code$ $gen(E.addr \neq E_1.addr '+' E_2.addr)$
$ - E_1$	$E.addr = \text{new Temp}()$ $E.code = E_1.code$ $gen(E.addr \neq \text{'minus' } E_1.addr)$
$ (E_1)$	$E.addr = E_1.addr$ $E.code = E_1.code$
$ \text{id}$	$E.addr = top.get(id.lexeme)$ $E.code = \text{''}$

Code attributes can be very long strings (as the expressions can be arbitrarily complex)

code 属性可以是很长的字符串（因为表达式可以任意复杂）

Incremental Translation Scheme

- In the SDT below, *gen* not only *generates* a three-address instruction, but also *appends* it to the sequence of instructions generated so far
- In comparison, in the previous SDD, the *code* attribute can be long strings after concatenations

$S \rightarrow \text{id} = E ;$	$\{ gen(top.get(id.lexeme) \neq E.addr); \}$
$E \rightarrow E_1 + E_2$	$\{ E.addr = \text{new Temp}();$ $gen(E.addr \neq E_1.addr '+' E_2.addr); \}$
$ - E_1$	$\{ E.addr = \text{new Temp}();$ $gen(E.addr \neq \text{'minus' } E_1.addr); \}$
$ (E_1)$	$\{ E.addr = E_1.addr; \}$
$ \text{id}$	$\{ E.addr = top.get(id.lexeme); \}$



Why this incremental approach can guarantee the correct order of instructions?

在下面的 SDT 中, *gen* 不仅生成三地址指令, 而且

还将其附加到迄今为止生成的指令序列中

相比之下, 在以前的 SDD 中 *code* 属性可以是串联后的长字符串

为什么这种增量方法可以保证指令的正确顺序?

code 属性可能是很长的字符串, 因此就像 5.5.2 节中讨论的那样, 它们通常是用增量的方式生成的。因此, 我们不会像图 6-19 所示的那样构造 *E.code*, 我们可以设法像图 6-20 中那样只生成新的三地址指令。在这个增量方式中, *gen* 不仅要构造出一个新的三地址指令, 还要将它添加到至今为止已生成的指令序列之后。指令序列可以暂时放在内存中以进一步处理, 也可以增量地输出。

Incremental Translation Scheme

- In the SDT below, *gen* not only *generates* a three-address instruction, but also *appends* it to the sequence of instructions generated so far
- In comparison, in the previous SDD, the *code* attribute can be long strings after concatenations

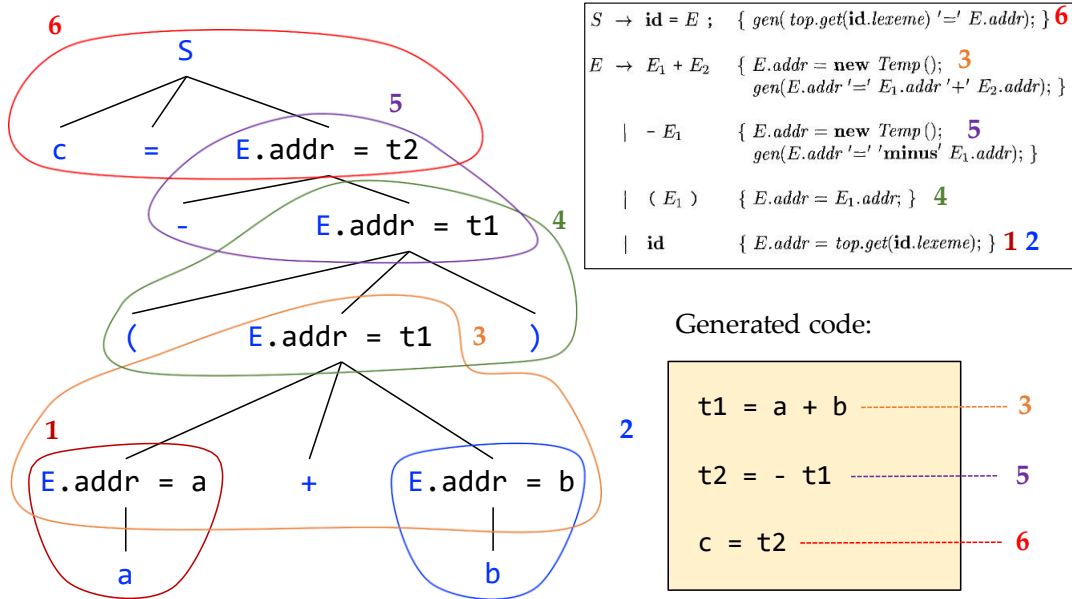
$S \rightarrow \text{id} = E ;$	$\{ \text{gen}(\text{top.get}(\text{id.lexeme}) \neq E.\text{addr}); \}$
$E \rightarrow E_1 + E_2$	$\{ E.\text{addr} = \text{new Temp}();$ $\text{gen}(E.\text{addr} \neq E_1.\text{addr} + E_2.\text{addr}); \}$
$\quad \quad - E_1$	$\{ E.\text{addr} = \text{new Temp}();$ $\text{gen}(E.\text{addr} \neq \text{'minus'} E_1.\text{addr}); \}$
$\quad \quad (E_1)$	$\{ E.\text{addr} = E_1.\text{addr}; \}$
$\quad \quad \text{id}$	$\{ E.\text{addr} = \text{top.get}(\text{id.lexeme}); \}$

This postfix SDT can be implemented in bottom-up parsing where subexpressions are always handled first (e.g., the code of E_1 and E_2 is generated before E)

采用增量方式时不需再用到 *code* 属性, 因为对 *gen* 的连续调用将生成一个指令序列。例如, 图 6-20 中对应于 $E \rightarrow E_1 + E_2$ 的语义规则直接调用 *gen* 产生一条加法指令。在此之前, 翻译方案已经生成了计算 E_1 的值并放入 $E_1.\text{addr}$ 、计算 E_2 的值并放入 $E_2.\text{addr}$ 的指令序列。

这种后缀 SDT 可以在自下而上的解析中实现, 其中总是首先处理子表达式 (例如, E_1 和 E_2 的 *code* 在 E 之前生成)

Example: Translating $c = -(a + b)$



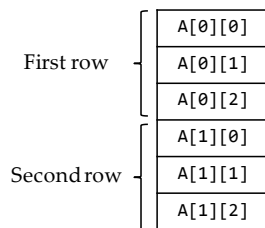
Fall 2024

BUPT Compilers

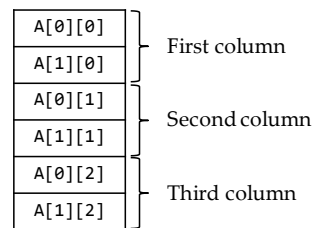
59

Addressing Array Elements

- Array elements can be accessed quickly if they are stored consecutively
- For an array A with n elements, the **relative address of $A[i]$** is:
 - $base + i * w$ ($base$ is the relative address of $A[0]$, w is the width of an element)
- For a 2D array A (row-major layout), the relative address of $A[i_1][i_2]$ is:
 - $base + i_1 * w_1 + i_2 * w_2$ (w_1 is the width of a row, w_2 is the width of an element)



Row-major (C)



Column-major (Fortran)

Fall 2024

BUPT Compilers

60

如果数组元素连续存储，则可以快速访问它们

对于阵列 A 具有 n 元素的相对地址 $A[i]$ 是:

$base + i * w$ ($base$ 是相对地址 $A[0]$, w 是元素的宽度)

对于二维阵列 A (行主布局) $A[i_1][i_2]$ 是:

$base + i_1 * w_1 + i_2 * w_2$ (w_1 是一行的宽度, w_2 是元素的宽度)

Addressing Array Elements

- Array elements can be accessed quickly if they are stored consecutively
- For an array A with n elements, the **relative address of $A[i]$** is:
 - $base + i * w$ ($base$ is the relative address of $A[0]$, w is the width of an element)
- For a 2D array A (row-major layout), the relative address of $A[i_1][i_2]$ is:
 - $base + i_1 * w_1 + i_2 * w_2$ (w_1 is the width of a row, w_2 is the width of an element)
- Further generalize to k -dimensional array A (row-major layout), the relative address of $A[i_1][i_2] \dots [i_k]$ is:
 - $base + i_1 * w_1 + i_2 * w_2 + \dots + i_k * w_k$ (w 's can be generalized as above)

如果数组元素连续存储, 则可以快速访问它们

对于阵列 A 具有 n 元素的相对地址 $A[i]$ 是: 一维数组: 假设每个数组元素的宽度是 w , 则数组元素 $a[i]$ 的相对地址是: $base + i \times w$

其中, $base$ 是数组的基地址, $i \times w$ 是偏移地址

$base + i * w$ ($base$ 是相对地址 $A[0]$, w 是元素的宽度)

对于二维阵列 A (行主布局) $A[i_1][i_2]$ 是: 假设一行的宽度是 w_1 , 同一行中每个数组元素的宽度是 w_2 , 则数组元素 $a[i_1][i_2]$ 的相对地址是:

$$base + i_1 \times w_1 + i_2 \times w_2$$

$base + i_1 * w_1 + i_2 * w_2$ (w_1 是一行的宽度, w_2 是元素的宽度)

进一步推广到 k -维度数组 A (行主布局) $A[i_1][i_2] \dots [i_k]$ 是:

$base + i_1 * w_1 + i_2 * w_2 + \dots + i_k * w_k$ (w 's 可以如上所述进行推广)

Translation of Array References

- The main problem in generating code for array references is to **relate the address-calculation formula to the grammar**
 - The relative address of $A[i_1][i_2] \dots [i_k]$ is $base + i_1 * w_1 + i_2 * w_2 + \dots + i_k * w_k$
 - Productions for generating array references: $L \rightarrow L[E] \mid \mathbf{id}[E]$

为数组引用生成代码的主要问题是将地址计算公式与语法联系起来

$A[i_1][i_2][i_k]$ 的相对地址是 $base + i_1 * w_1 + i_2 * w_2 + \dots + i_k * w_k$

用于生成数组引用的产生式: $L \rightarrow L[E] \mid \mathbf{id}[E]$

➤ 假设 $\text{type}(a) = \text{array}(3, \text{array}(5, \text{array}(8, \text{int})))$,
 一个整型变量占用4个字节,

$$\begin{aligned} \text{则 } \text{addr}(a[i_1][i_2][i_3]) &= \text{base} + i_1 * w_1 + i_2 * w_2 + i_3 * w_3 \\ &= \text{base} + i_1 * 160 + i_2 * 32 + i_3 * 4 \end{aligned}$$

$a[i_1]$ 的宽度 ————
 $a[i_1][i_2]$ 的宽度 ————
 $a[i_1][i_2][i_3]$ 的宽度 ————

SDT for Array References (1)

```

L → id [ E ] { L.array = top.get(id.lexeme);
               L.type = L.array.type.elem;
               L.addr = new Temp();
               gen(L.addr '=' E.addr '*' L.type.width); }

| L1 [ E ] { L.array = L1.array;
              L.type = L1.type.elem;
              t = new Temp();
              L.addr = new Temp();
              gen(t '=' E.addr '*' L.type.width);
              gen(L.addr '=' L1.addr '+' t); }
  
```

$L.array$: a pointer to the symbol-table entry for the array name

$L.array.base$: the base address of the array

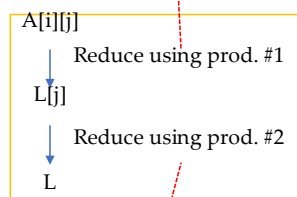
$L.addr$: a temporary for computing the offset for the array reference

$L.type$: the type of the subarray generated by L

$t.elem$: for any array type t , $t.elem$ gives the element type

A is a 2*3 array of integers

$L.type$ is the type of A's element:
 $\text{array}(3, \text{int})$



$L.type$ is the type of A[i]'s element:
 int

与 C 和 Java 中一样，我们假定数组元素的最小编号是 0。我们使用式(6.4)，基于宽度来计算相对地址，而不是像式(6.6)中那样使用元素的数量来计算地址。图 6-22 所示的翻译方案为带有数组引用的表达式生成三地址代码。它包括了图 6-20 中给出的产生式和语义动作，同时还包括了涉及非终结符号 L 的产生式。

非终结符号 L 有三个综合属性：

L.array 是一个指向数组名字对应的符号表条目的指针。在分析了所有的下标表达式之后，该数组的基地址，也就是 L.array.base,被用于确定一个数组引用的实际左值。

L.addr 指示一个临时变量。这个临时变量将被用于累加公式(6.4)中的 $i_j \times w_1$ 项，从而计算数组引用的偏移量。

L.type 是 L 生成的子数组的类型。对于任何类型 t，我们假定其宽度由 t.width 给出。我们把类型（而不是宽度）作为属性，是因为无论如何类型检查总是需要这个类型信息。

对于任何数组类型 t，假设 t.elem 给出了其数组元素的类型。

SDT for Array References (2)

- The semantic actions of L-productions compute offsets
- The address of an array element is $base + offset$

$$\begin{aligned}
 E \rightarrow E_1 + E_2 & \quad \{ E.addr = \text{new Temp}(); \\
 & \quad \quad \quad gen(E.addr := E_1.addr + E_2.addr); \} \\
 | \text{ id} & \quad \{ E.addr = top.get(id.lexeme); \} \\
 | L & \quad \{ E.addr = \text{new Temp}(); \\
 & \quad \quad \quad gen(E.addr := L.array.base + L.addr); \}
 \end{aligned}$$

Instruction of the form $x = a[i]$

Array references can be part of an expression

L-产生式的语义动作计算偏移

数组元素的地址为 $base + offset$

产生式 $E \rightarrow E1 + E2$ 和 $E \rightarrow id$ 与以前相同。新的产生式 $E \rightarrow L$ 的语义动作生成的代码将 L 所指位置上的值复制到一个新的临时变量中。和前面对产生式 $S \rightarrow L = E$ 的讨论一样, L 所指的地址就是 $L.array.base[L.addr]$ 。其中, 属性 $L.array$ 仍然给出了数组名, $L.array.base$ 给出了数组的基地址。属性 $L.addr$ 表示保存偏移量的临时变量。数组引用的代码将存放在由基地址和偏移量给出的位置中的右值放入 $E.addr$ 所指的临时变量中。

SDT for Array References (3)

$$S \rightarrow id = E ; \quad \{ gen(top.get(id.lexeme) \neq E.addr); \}$$

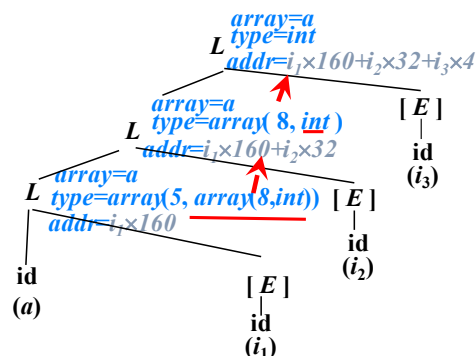
$$| \quad L = E ; \quad \{ gen(L.addr.base'[L.addr]' \neq E.addr); \}$$

Instruction of form $a[i] = x$

Array references can appear at the LHS of an assignment statement

产生式 $S \rightarrow id = E$; 代表一个对非数组变量的赋值语句, 它按照通常的方法进行处理。

$S \rightarrow L = E$; 的语义动作产生了一个带下标的复制指令, 它将表达式 E 的值存放到数组引用 L 所指的内存位置。回顾一下, 属性 $L.array$ 给出了数组的符号表条目。数组的基地址 (即 0 号元素的地址) 由 $L.array.base$ 给出。属性 $L.addr$ 表示一个临时变量, 它保存了 L 生成的数组引用的偏移量。因此, 这个数组引用的位置是 $L.array.base[L.addr]$ 。这个指令将地址 $E.addr$ 中的右值放入 L 的内存位置中。



$$base + i_1 \times w_1 + i_2 \times w_2 + \dots + i_k \times w_k$$

假设 $type(a) = array(3, array(5, array(8, int)))$, 计算 $a[i_1][i_2][i_3]$ 数组引用的偏移地址

➤ L 的综合属性

➤ $L.type$: L 生成的数组元素的类型（数组类型的表达式：二元组（元素数量，元素类型））

➤ $L.addr$: 指示一个临时变量，该临时变量用于累加公式中的 $i_j \times w_j$ 项，从而计算数组引用的偏移量

➤ $L.array$: 数组名在符号表的入口地址

翻译的关键：地址计算公式与数组引用的文法关联起来

简单赋值语句的翻译采用的是自底向上的翻译方法，这里还可以延续该方法。

从左向右扫描，查询符号表得到 a 的类型表达式和它的基地址，base 和 i1 的值就可以确定了，接下来求 w1（a[i1] 的宽度），由 a 的类型表达式即知道其元素 a[i1] 的类型表达式，设置 L.type 来表示，计算出 a[i1] 的宽度，设置 L.offset 偏移地址，接下来 a[i1][i2] 是 a[i1] 的一个元素，可通过子节点 a[i1] 元素的类型表达式获得 a[i1][i2] 的类型表达式，进而得到其宽度 w2，接下来计算累计的偏移地址 L.offset

数组引用的 SDT

假设 $type(a) = array(3, array(5, array(8, int)))$,
翻译语句片段 “ $a[i_1][i_2][i_3]$ ”

$$addr(a[i_1][i_2][i_3]) = base + i_1 \times w_1 + i_2 \times w_2 + i_3 \times w_3$$

```

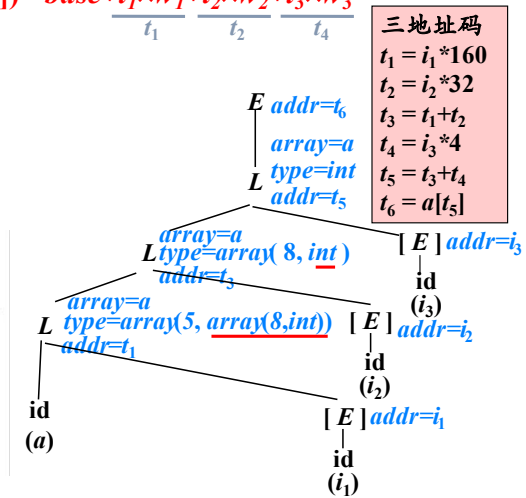
E → E1 + E2  { E.addr = new Temp();
                  gen(E.addr := E1.addr + E2.addr); }

| id             { E.addr = top.get(id.lexeme); }

| L             { E.addr = new Temp();
                  gen(E.addr := L.array.base + L.addr); }

L → id [ E ]    { L.array = top.get(id.lexeme);
                  L.type = L.array.type.elem;
                  L.addr = new Temp();
                  gen(L.addr := E.addr * L.type.width); }

| L1 [ E ]    { L.array = L1.array;
                  L.type = L1.type.elem;
                  t = new Temp();
                  L.addr = new Temp();
                  gen(t := E.addr * L.type.width);
                  gen(L.addr := L1.addr + t); }
    
```



```

E → E1 + E2  { E.addr = new Temp(); 6
                  gen(E.addr := E1.addr + E2.addr); }

| id             { E.addr = top.get(id.lexeme); } 1 3

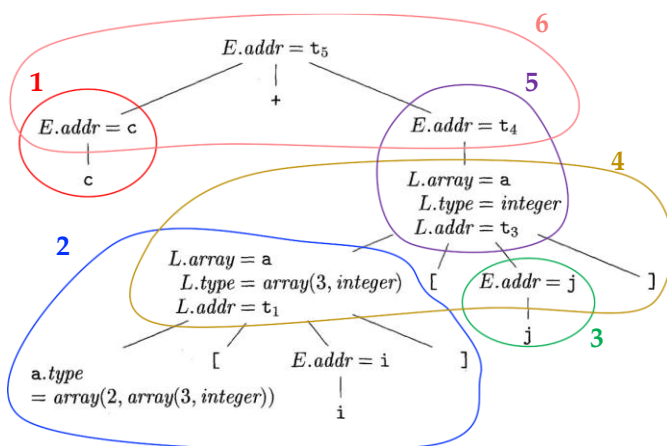
| L             { E.addr = new Temp(); 5
                  gen(E.addr := L.array.base + L.addr); }
    
```

```

L → id [ E ]    { L.array = top.get(id.lexeme); 2
                  L.type = L.array.type.elem;
                  L.addr = new Temp();
                  gen(L.addr := E.addr * L.type.width); }

| L1 [ E ]    { L.array = L1.array;
                  L.type = L1.type.elem; 4
                  t = new Temp();
                  L.addr = new Temp();
                  gen(t := E.addr * L.type.width);
                  gen(L.addr := L1.addr + t); }
    
```

Translating $c + a[i][j]$



Generated code:

```

t1 = i * 12 2
t2 = j * 4 4
t3 = t1 + t2
t4 = a[t3] 5
t5 = c + t4 6
    
```

L 的综合属性:

L.array: 数组名在符号表的入口地址

L.type: L 生成的数组元素的类型

L.Addr: 指示一个临时变量, 该临时变量用于累加公式中的 $ij \times w_j$ 项, 从而计算数组引用的偏移量

Type Checking

- To do type checking, a compiler needs to assign a **type expression** to each component of the source program
- The compiler then determines that the type expressions conform to a **collection of logical rules** (i.e., the **type system**)
 - A **sound** type system allows us to determine statically that type errors cannot occur at run time
- A language is **strongly typed** if the compiler guarantees that the programs it accepts will run without type errors (**sound type system**)
 - **Strongly typed**: Java (`double a; int b = a; //cannot compile`)
 - **Weakly typed**: C/C++ (`double a; int b = a; //implicit conversion`)

要进行类型检查, 编译器需要为源程序的每个组件分配一个类型表达式

编译器然后确定类型表达式符合逻辑规则的集合 (即类型系统)

健全的类型系统允许我们静态地确定类型错误不会在运行时发生

如果编译器保证其接受的程序在运行时不会出现类型错误, 则语言是强类型的 (健全的类型系统)

强类型: Java (`double a; int b=a; //无法编译`)

弱类型: C/C++ (`double a; int b=a; //隐式转换`)

为了进行类型检查(type checking), 编译器需要给源程序的每一个组成部分赋予一个类型表达式。然后, 编译器要确定这些类型表达式是否满足一组逻辑规则。这些规则称为源语言的类型系统(type system)。

类型检查具有发现程序中的错误的潜能。原则上，如果目标代码在保存元素值的同时保存了元素类型的信息，那么任何检查都可以动态地进行。一个健全(sound)的类型系统可以消除对动态类型错误检查的需要，因为它可以帮助我们静态地确定这些错误不会在目标程序运行的时候发生。如果编译器可以保证它接受的程序在运行时刻不会发生类型错误，那么该语言的这个实现就被称为强类型的。

除了用于编译，类型检查的思想还可以用于提高系统的安全性，使得人们安全地导入和执行软件模块。Java 程序被编译成为机器无关的字节码，在字节码中包含了有关字节码中的运算的详细类型信息。导入的代码在被执行之前首先要进行类型检查，以防止因疏忽造成的错误和恶意攻击。

Rules for Type Checking

- **Typesynthesis (类型合成)**

- Build up the type of an expression from the types of subexpressions
 - **Typicalform:** if f has type $s \rightarrow t$ and x has type s , then expression $f(x)$ has type t
 - **Example:** $f(x) = -x$ (can be generalized to multi-argument cases)

- **Typeinference (类型推导)**

- Determine the type of a language construct from the way it is used
 - **Typicalform:** if $f(x)$ is an expression, then for some types α and β , f has type $\alpha \rightarrow \beta$ and x has type α
 - **Example:** let $null$ be a function that tests whether a list is empty, then from the usage $null(x)$, we can tell that x must be a list

类型合成：

根据子表达式的类型建立表达式的类型

典型形式：如果 f 具有类型 $s \rightarrow t$ 和 x 具有类型 s ，那么表达式 $f(x)$ 具有类型 t

示例： $f(x) = -x$ （可以推广到多参数的情况）

类型推导：

根据语言结构的使用方式确定其类型

典型形式：如果 $f(x)$ 是表达式，则对于某些类型 α 和 β , f 具有类型 $\alpha \rightarrow \beta$ 和 x 具有类型 α

示例： `null` 是一个测试列表是否为空的函数，然后从用法 `null(x)`，我们可以看出 x 必须是列表

Type Conversions

- Consider an expression $x * i$, where x is a float and i is an integer
 - The representation (the way of organizing 0/1 bits) of integers and floating-point numbers is different
 - Different machine instructions are used for operations on integers and floats
 - Convert integers to floats: $t_1 = (\text{float}) i$ $t_2 = x \text{ fmul } t_1$
- **Type conversion SDT** for a simple case (using type synthesis)
 - $E \rightarrow E_1 + E_2$
 - ```
{ if($E_1.type = integer$ and $E_2.type = integer$) $E.type = integer$;
 else if($E_1.type = float$ and $E_2.type = integer$) $E.type = float$;
 ...
}
```

Fall 2024

BUPT Compilers

73

考虑类似于  $x + i$  的表达式，其中  $x$  是浮点数类型而  $i$  是整型。因为整数和浮点数在计算机中有不同的表示形式，而且使用不同的机器指令来完成整数和浮点数运算 C 编译器需要把  $+$  的某个运算分量进行转换，以保证在进行加法运算时两个运算分量具有相同的类型。

假定在必要的时候可以使用一个单目运算符 `(float)` 将整数转换成浮点数 例如，整数 2 在表达式 `2 * 3.14` 对应的代码中被转换成浮点数：

```
t1 = (float) 2
```

```
t2 = t1 * 3.14
```

我们可以扩展这样例子，考虑运算符的整型和浮点型版本。比如， `int *` 表示作用于整型运算分量的运算符，而 `float *` 表示作用于浮点型运算分量的运算符。

我们将扩展 6.4.2 节中的用于表达式翻译的翻译方案，以说明如何进行类型综合。我们引入

另一个属性  $E.type$ , 该属性的值可以是 `integer` 或 `float`。和  $E \rightarrow E1 + E2$  相关的规则可用如下的伪代码给出：

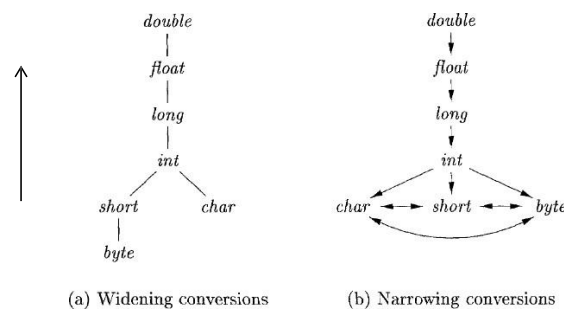
简单情况的类型转换 SDT（使用类型合成）

$E \rightarrow E1 + E2.$

```
{if($E1.type = integer$ and $E2.type = integer$) $E.type = integer$;
else if $E1.type = float$ and $E2.type = integer$ $E.type = float$;
...
}
```

## Widening and Narrowing (1)

- Type conversion rules vary from language to language
- Java distinguishes between *widening* conversions (类型拓宽) and *narrowing* conversions (类型窄化)

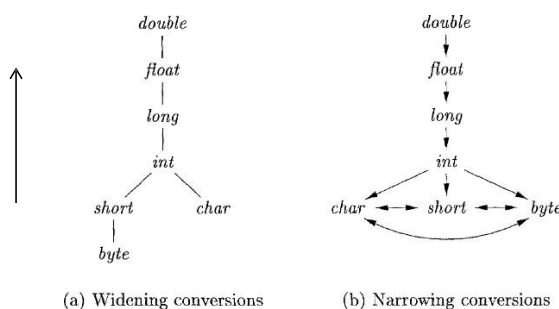


随着需要转换的类型的增多，需要处理的不同情况也急剧增多。因此，在处理大量的类型时，精心组织用于类型转换的语义动作就变得非常重要。

不同语言具有不同的类型转换规则。图 6-25 中的 Java 的转换规则区分了拓宽(widening)转换和窄化(narrowing)转换。

## Widening and Narrowing (2)

- **Widening** conversions **preserve information** and can be done automatically by the compiler (*implicit* type conversions, or *coercions*)
- **Narrowing** conversions **lose information** and require programmers to write code to cause the conversion (*explicit* type conversions, or *casts*)



拓宽转换可以保持原有的信息，而窄化转换则可能丢失信息。

拓宽规则通过图 6-25a 中的层次结构给出：在该层次结构中位于较低层的类型可以被拓宽为较高层的类型。因此，char 类型可以被拓宽为 int 型和 float 型，但是不可以被拓宽为 short 类型。

窄化转换的规则如图 6-25b 所示：如果存在一条从 s 到 t 的路径，则可以将类型 s 窄化为类型 t。可以看出，char、short、byte 之间可以两两相互转换。

如果类型转换由编译器自动完成，那么这样的转换就称为隐式转换。隐式转换也称为自动类型转换(coercion)。在很多语言中，自动类型转换仅仅限于拓宽转换。如果程序员必须写出某些代码来引发类型转换运算，那么这个转换就称为显式的。显式转换也称为强制类型转换(cast)。

# SDT for Type Conversion

- $\text{max}(t_1, t_2)$  takes two types  $t_1$  and  $t_2$  and returns the **maximum** (or **least upper bound**) of the two types in the widening hierarchy
- $\text{widen}(a, t, w)$  generates type conversions if needed to widen an address  $a$  of type  $t$  into a value of type  $w$

```
Addr widen(Addr a, Type t, Type w)
{
 if (t = w) return a;
 else if (t = integer and w = float) {
 temp = new Temp();
 gen(temp := (float) a);
 return temp;
 }
 else error;
}
```

```
E → E1 + E2 { E.type = max(E1.type, E2.type);
 a1 = widen(E1.addr, E1.type, E.type);
 a2 = widen(E2.addr, E2.type, E.type);
 E.addr = new Temp();
 gen(E.addr := a1 + a2); }
```

检查  $E \rightarrow E_1 + E_2$  的语义动作使用了两个函数：

1)  $\text{max}(t_1, t_2)$  接受  $t_1$  和  $t_2$  两个类型的参数并返回拓宽层次结构中这两个类型中的最大者（或者最小上界）。如果  $t_1$  或  $t_2$  之一没有出现在这个

层次结构中，比如有个类型是数组类型或指针类型，那么该函数返回一个错误信息。

2) 如果需要将类型为  $t$  的地址  $a$  中的内容转换成  $w$  类型的值，则函数  $\text{widen}(a, t, w)$  将生成类型转换的代码。如果  $t$  和  $w$  是相同的类型，则该函数返回  $a$  本身。否则，它会生成一条指令来完成转换工作并将转换结果放置到临时变量  $\text{temp}$  中。这个临时变量将作为结果返回。函数  $\text{widen}$  的伪代码如图 6-26 所示，这里假设只有 `integer` 和 `float` 两种类型。

# Example

- $a + b$  (suppose  $a$  is of *int* type and  $b$  is of *float* type)

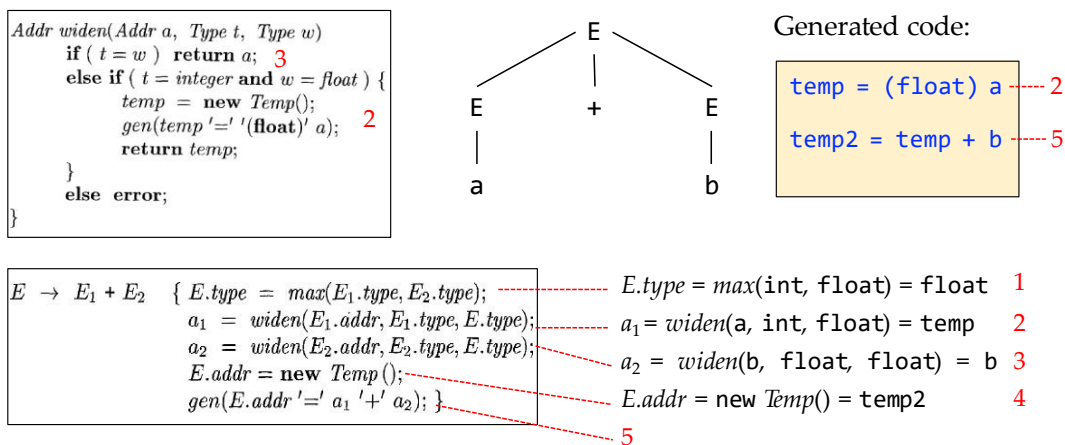


图 6-27 中  $E \rightarrow E_1 + E_2$  的语义动作说明了如何把类型转换加入到图 6-20 所示的翻译表达式的方案中。

在这个语义动作中，如果  $E_1$  的类型不需要被转换成  $E$  的类型，那么临时变量  $a_1$  就是  $E_1.addr$ 。如果需要进行这样的转换，则  $a_1$  就是 `widen` 函数返回的一个新的临时变量。类似地， $a_2$  可能是  $E_2.addr$ ，也可能是一个新临时变量，用于存放转换后的  $E_2$  的值。如果两个变量都是整型或者都是浮点型，就不需要进行任何转换。我们会发现，将两个不同类型的值相加的唯一方法是把它们都转换成为第三种类型。

# Control Flow

- Boolean expressions are often used to [alter the flow of control](#) or [compute logical values](#)
- **Grammar:**  $B \rightarrow B \parallel B \mid B \&\& B \mid !B \mid (B) \mid E \text{ rel } E \mid \text{true} \mid \text{false}$
- Given the expression  $B_1 \parallel B_2$ , if  $B_1$  is true, then the expression is true without having to evaluate  $B_2$ . Thus, neither  $B_1$  nor  $B_2$  is necessarily evaluated fully.\*
- In *short-circuit code*, the boolean operators  $\&\&$ ,  $\parallel$ ,  $!$  translate into jumps. [The operators do not appear in the code.](#)

If  $B_1$  or  $B_2$  has side effect (e.g., changing the value of a global variable), then the effect may not occur

在编程语言中，布尔表达式通常用于

1. 改变控制流。 Boolean 表达式在改变控制流的语句中用作条件表达式。

这种布尔表达式的值由程序到达的某个位置隐含地指出。例如，在  $\text{if}(E) S$  中，如果运行到语句  $S$ ，则表达式  $E$  必须为 `true`。

2. 计算逻辑值。一个布尔表达式的值可以表示为 `true` 或 `false`。这样的布尔表达式也可以像算术表达式一样，使用带有逻辑运算符的三地址指令进行求值。

布尔表达式的使用意图，要根据其语法上下文决定。例如，关键字 `if` 后面的表达式用于更改控制流，而一个赋值语句右侧的表达式用于表示一个逻辑值。这样的语法上下文可以通过多种方式来指定：我们可以使用两个不同的非终结符，也可使用继承的属性，还可以在语法分析过程中设置一个标记。此外，我们还可以构建一个语法树并调用不同的过程来处理布尔表达式的两种不同的使用。

本节将介绍用于改变控制流的布尔表达式，为此引入一个新的非终结符  $B$ 。布尔表达式是将作用布尔变量或关系表达式的布尔运算符构成的。我们使用 C 语言的方法，用  $\&\&$ ， $\parallel$ ， $!$  分别表示 AND，OR，NOT 运算符。关系表达式的形式为  $E \text{ rel } E$ 。其中， $E$  为算术表达式。写出布尔表达式的文法：

$B \rightarrow B \ \&\& \mid B \ \&\& \ B \mid ! \ (B) \mid E \ \text{rel} \ E \mid \text{true} \mid \text{false}$

我们通过属性 `rel.op` 来指明 `rel` 究竟表示 6 种比较运算符 `<`, `<=`, `=`, `!=`, `>` 和 `>=` 中的哪一种。按照惯例, 假设 `||` 和 `&&` 是左结合的, `||` 的优先级最低, 再其次为!

给定表达式  $B1 \ \&\& \ B2$ , 如果  $B1$  为真, 那么无需对  $B2$  求值, 表达式也为真。因此,  $B1$  和  $B2$  都不一定要完全求值\*。

程序设计语言的语义定义决定是否需要对于一个布尔表达式的各个部分都进行求值。如果语言定义允许 (或要求) 不对布尔表达式的某部分求值, 则编译器可以优化布尔表达式的求值过程, 只要已经求值的部分足以确定整个表达式的值就可以了。因此, 在诸如  $B1 \ || \ B2$  之类的表达式中  $B2$ ,  $B1$  和  $B2$  都不必完全地求值。如果  $B1$  或  $B2$  是具有副作用的表达式 (例如, 它包含了改变一个全局变量的函数), 那这么做就可能会得到意料之外的结果。

在短路 (跳转) 代码中, 布尔运算符 `&&`、`||`、`!` 转化为跳转。这些运算符本身不出现在代码中, 布尔表达式的值是通过代码序列中的位置来表示的。

## Short-Circuit Code Example

• `if ( x < 100 || x > 200 && x != y ) x = 0;`

```
 if x < 100 goto L2
 ifFalse x > 200 goto L1
 ifFalse x != y goto L1
L2: x = 0
L1:
```

上述语句可以翻译成图 6.34 的代码。在此翻译中, 如果布尔表达式为 `true`, 则控制到达 `L2`。如果表达式为 `false`, 则控制将跳转过 `L2` 和赋值语句 `x = 0`, 直接转到 `L1`。

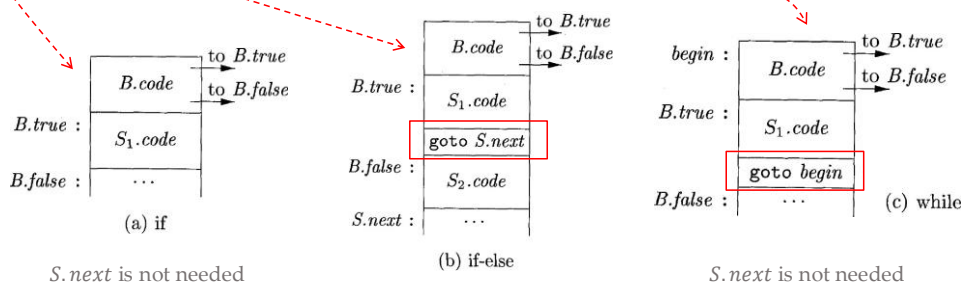
# Flow-of-Control Statements

- Grammar:

- $S \rightarrow \text{if } (B) S_1$
- $S \rightarrow \text{if } (B) S_1 \text{ else } S_2$
- $S \rightarrow \text{while } (B) S_1$

**Inherited attributes:**

- $B.true$ : the label to which control flows if  $B$  is true
- $B.false$ : the label to which control flows if  $B$  is false
- $S.next$ : the label for the instruction immediately after the code for  $S$



Fall 2024

BUPT Compilers

81

现在我们考虑在语句上下文中如何将布尔表达式转换为三地址代码。

在这些产生式中，非终结符  $B$  表示布尔表达式，非终结符  $S$  表示语句。

这个文法将之前我们在语法制导翻译中（例 5.19）所介绍的 `while` 表达式进行了推广。和那个例子一样， $B$  和  $S$  都有一个综合属性 `code`，该属性给出了翻译得到的三地址指令。为简单起见，我们使用语法制导定义来构造得到翻译结果 `B.code` 和 `S.code`，结果值是字符串。定义了 `code` 属性的语义规则可以按照如下的方法来实现：首先构造语法树，然后在树遍历期间产生目标代码。这些规则还可通过第 5.5 节中列出的任何方法来实现。

如图 6.35(a) 所示，对 `if(B) S1` 的翻译结果中包含了 `B:code`，其后是 `S1.code`。

`B.code` 中存在基于  $B$  值的跳转。如果  $B$  为真，则控制流转向 `S1.code` 的第一条指令；如果  $B$  为假，则控制流立即转向紧随在 `S1.code` 之后的指令。`B.code` 和 `S.code` 中跳转的标号使用继承属性来处理。我们将布尔表达式  $B$  和两个标号 `B:true` 和 `B:false` 相关联。如果  $B$  为 `true`，则控制流向 `B:true`；如果  $B$  为 `false`，则控制流转到 `B:false`。对于语句  $S$ ，我们关联一个继承属性 `S.next`，表示紧接在  $S$  的代码之后的指令的标号。在某些情况下，紧跟在 `S.code` 后面的指令是跳转到某个标号  $L$  的跳转指令。使用 `S.next` 可以避免在 `S.code` 中出现这样一条跳转指令，它的目标又是一个以  $L$  为目标的跳转指令。



## SDD for Flow-of-Control Statements (1)

| PRODUCTION                                            | SEMANTIC RULES                                                                                                                                                                                                                            |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $P \rightarrow S$                                     | $S.next = newlabel()$<br>$P.code = S.code \parallel label(S.next)$                                                                                                                                                                        |
| $S \rightarrow \text{assign}$                         | $S.code = \text{assign.code}$                                                                                                                                                                                                             |
| $S \rightarrow \text{if} ( B ) S_1$                   | $B.true = newlabel()$<br>$B.false = S_1.next = S.next$<br>$S.code = B.code \parallel label(B.true) \parallel S_1.code$                                                                                                                    |
| $S \rightarrow \text{if} ( B ) S_1 \text{ else } S_2$ | $B.true = newlabel()$<br>$B.false = newlabel()$<br>$S_1.next = S_2.next = S.next$<br>$S.code = B.code$<br>$\parallel label(B.true) \parallel S_1.code$<br>$\parallel gen('goto' S.next)$<br>$\parallel label(B.false) \parallel S_2.code$ |

This SDD is neither S-attributed nor L-attributed.  
Computing the attributes requires a separate pass after parsing.

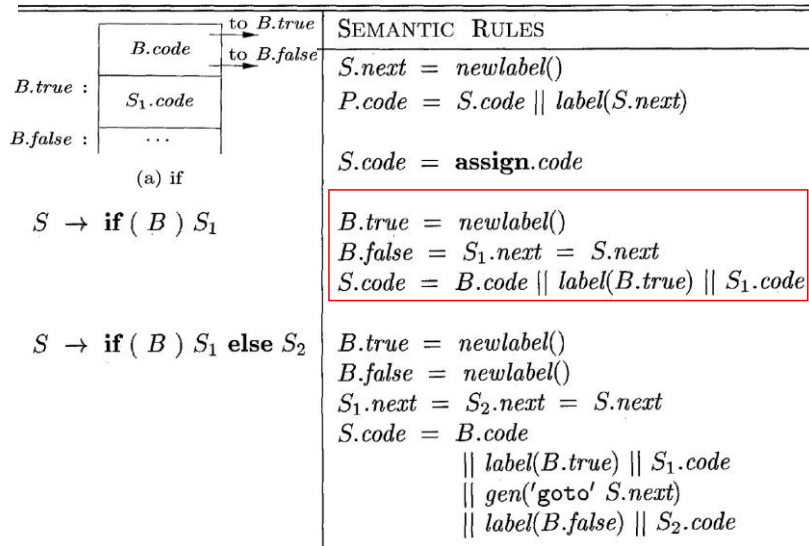
图 6.36 和 6.37 给出的语法制导定义可以为 if、if-else 和 while 语句的上下文中为布尔表达式生成三地址代码。

我们假设 new label() 每次被调用时都会创建一个新的标号，并且 label(L) 将标号 L 附加到下一个要生成的三地址指令上。

- 一个程序包含一条由产生式  $P \rightarrow S$  生成的语句组成。与该产生式相关的语法规则将 S:next 初始化为一个新的标号。P:code 包含 S:code，S:code 后跟新标签 S:next 组成。

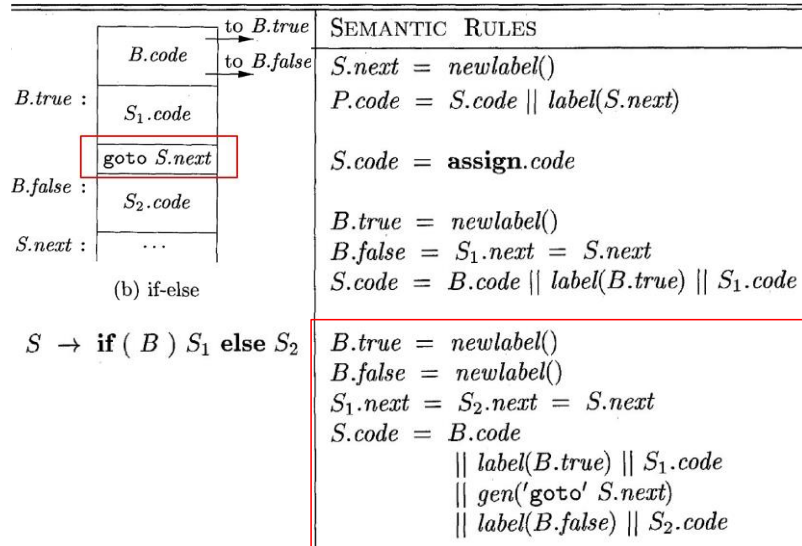
- 产生式  $S \rightarrow \text{assign}$  中的词法单元 assign 是一个表示赋值语句的占位符。赋值语句的翻译如第 6.4 节所述；在这里对控制流的讨论中，S:code 就是 assign.code。

## SDD for Flow-of-Control Statements (1)



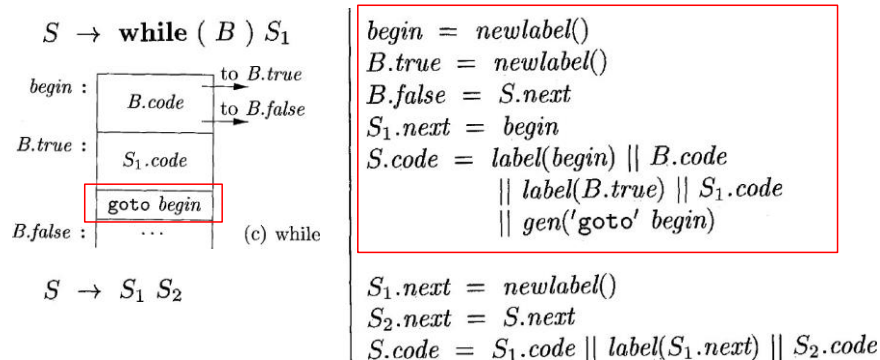
- 在翻译  $S \rightarrow \text{if}(B) S_1$  时，图 6.36 中的语义规则创建一个新的标号  $B.true$ ，并将其关联到为语句  $S_1$  生成的第一个三地址指令中，如图 6.35(a) 所示。因此，在  $B$  的代码中跳转到  $B.true$  的指令跳转到语句  $S_1$  对应的代码。此外，通过将  $B.false$  设置为  $S.next$ ，我们确保当  $B$  的值为  $false$ ，控制将跳过  $S_1$  的代码。

## SDD for Flow-of-Control Statements (1)



在翻译 if-else-语句  $S \rightarrow \text{if}(B) S_1 \text{ else } S_2$  时，如果  $B$  为真，则布尔表达式  $B$  的代码跳至  $S_1$  代码的第一条指令，如果  $B$  为假，则跳至  $S_2$  代码的第一条指令，如图 6.35(b)所示。此外，控制从  $S_1$  或  $S_2$  流向紧跟在  $S$  代码之后的三地址指令——该指令的标号由继承属性  $S.next$  给出。在  $S_1$  的代码之后有一条 goto  $S.next$  指令，使得控制流跳过  $S_2$  的代码。 $S_2$  之后不需要 goto，因为  $S_2.next$  就是  $S.next$ 。

## SDD for Flow-of-Control Statements (2)

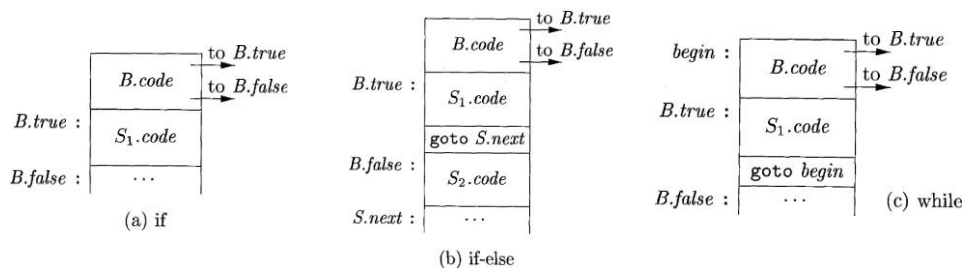


•  $S \rightarrow \text{while} ( B ) S_1$  的代码由 `B:code` 和 `S1:code` 组成, 如图 6.35(c)所示。我们使用一个局部变量 `begin` 来保存附加到该 `while` 语句的第一条指令上的标号。这个 `while` 语句的第一条指令也是 `B` 的第一条指令。我们这里使用变量而不是属性, 是因为 `begin` 对于这个产生式来说是局部的语义规则。继承属性 `S:next` 标记了当 `B` 为 `false` 时控制必须转向的标号; 因此, `B:false` 被设置为 `S:next`。在 `S1` 的第一条指令上附加了一个新的标号 `B:true`。 `B` 的指令中的跳转指令在 `B` 为真时跳转到这个标号。我们在 `S1` 的代码之后, 放置了一条 `goto begin` 指令, 它跳转到布尔表达式的代码的开始处。请注意, `S1:next` 被设置标号 `begin`, 因此从 `S1:code` 中跳出的指令可以直接跳转 `begin`。

•  $S \rightarrow S_1 S_2$  的代码由 `S1` 代码和 `S2` 组成。相应的语义规则主要处理标号。`S1` 代码之后的第一条指令就是 `S2` 代码的起始指令。紧跟在 `S2` 代码之后的指令也是跟在 `S` 的代码之后的指令。

## Translating Boolean Expressions in Flow-of-Control Statements

- A boolean expression  $B$  is translated into three-address instructions that evaluate  $B$  using conditional and unconditional jumps to one of two labels:  $B.true$  and  $B.false$ 
  - $B.true$  and  $B.false$  are two inherited attributes. Their value depends on the context of  $B$  (e.g., *if* statement, *if-else* statement, *while* statement)



如图 6.35 的代码布局所示，布尔表达式  $B$  被转换为三地址指令，它使用有条件和无条件跳转指令来对  $B$  求值。这些跳转指令的目标是两个标号之一：如果  $B$  为 `true`，则  $B.true$ ；如果  $B$  为 `false`，则  $B.false$ 。

$B.true$  和  $B.false$  是两个继承属性。它们的值取决于的上下文（如 *if* 语句、*if-els* 语句、*wile* 语句）

# Generating Three-Address Code for Booleans (1)

| PRODUCTION                       | SEMANTIC RULES                                                                                                                                                                     |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $B \rightarrow B_1 \    \ B_2$   | $B_1.true = B.true$ // short-circuiting<br>$B_1.false = newlabel()$<br>$B_2.true = B.true$<br>$B_2.false = B.false$<br>$B.code = B_1.code \    \ label(B_1.false) \    \ B_2.code$ |
| $B \rightarrow B_1 \ \&\& \ B_2$ | $B_1.true = newlabel()$<br>$B_1.false = B.false$ // short-circuiting<br>$B_2.true = B.true$<br>$B_2.false = B.false$<br>$B.code = B_1.code \    \ label(B_1.true) \    \ B_2.code$ |
| $B \rightarrow ! \ B_1$          | $B_1.true = B.false$<br>$B_1.false = B.true$<br>$B.code = B_1.code$                                                                                                                |

图 6.37 中针对布尔表达式的语义规则是图 6.36 中语句的语义规则的一个补充。

B 的其余产生式翻译如下：

- 假设 B 的形式为 B1 || B2.如果 B1 为真，那么我们立即知道 B 本身为真，因此 B1:true 与 B:true 相同。如果 B1 为 false，则必须计算 B2，因此我们将 B1:false 设为 B2 代码中第一条指令的标号。 B2 的真出口和假出口分别与 B 的真出口和假出口相同。
- B1 && B2 的翻译类似。
- 不需要为 B →! B1 产生新的代码，只需将 B 的真假出口互换即可得到 B1 的真假出口。

# Generating Three-Address Code for Booleans (2)

|                                      |                                                                                                                                                    |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| $B \rightarrow E_1 \text{ rel } E_2$ | $B.code = E_1.code \parallel E_2.code$<br>$\parallel gen('if' E_1.addr \text{ rel } op E_2.addr 'goto' B.true)$<br>$\parallel gen('goto' B.false)$ |
| $B \rightarrow \text{true}$          | $B.code = gen('goto' B.true)$                                                                                                                      |
| $B \rightarrow \text{false}$         | $B.code = gen('goto' B.false)$                                                                                                                     |

图 6.37 中的第四个产生式， $B \rightarrow E_1 \text{ rel } E_2$ ，直接翻译成三地址比较指令，并跳转到适当的位置。例如， $a < b$  形式的  $B$  转换为：

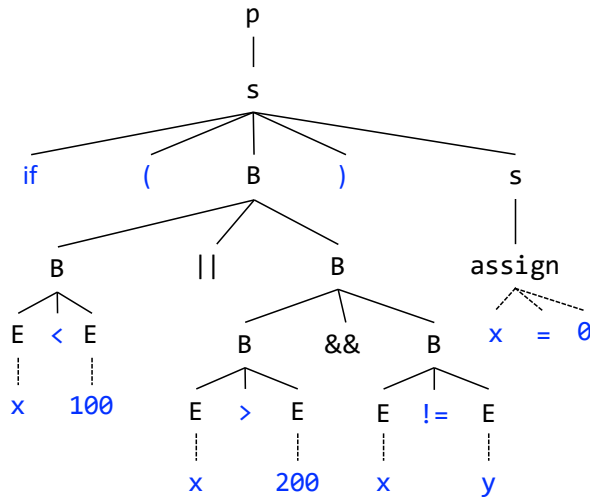
if  $a < b$  goto B:true

goto B:false

将常量 `true` 和 `false` 分别翻译成目标为 B:true 和 B:false 的跳转指令。

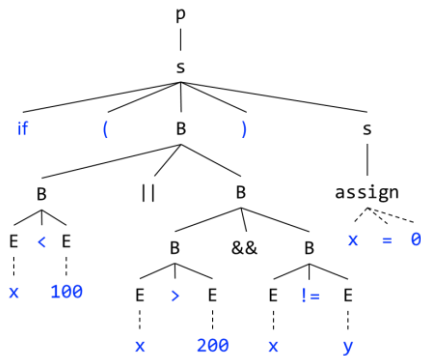
# Example

• `if ( x < 100 || x > 200 && x != y ) x = 0;`



Dashed lines mean that the reduction may consist of multiple steps

# Example



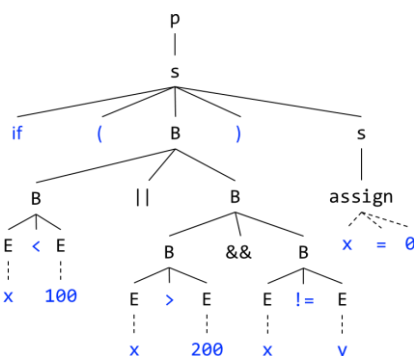
| PRODUCTION                           | SEMANTIC RULES                                                                                                                                                       |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $P \rightarrow S$                    | $S.next = newlabel()$<br>$P.code = S.code \parallel label(S.next)$                                                                                                   |
| $S \rightarrow assign$               | $S.code = assign.code$                                                                                                                                               |
| $S \rightarrow if ( B ) S_1$         | $B.true = newlabel()$<br>$B.false = S_1.next = S.next$<br>$S.code = B.code \parallel label(B.true) \parallel S_1.code$                                               |
| $B \rightarrow B_1 \parallel B_2$    | $B_1.true = B.true$<br>$B_1.false = newlabel()$<br>$B_2.true = B.true$<br>$B_2.false = B.false$<br>$B.code = B_1.code \parallel label(B_1.false) \parallel B_2.code$ |
| $B \rightarrow B_1 \&\& B_2$         | $B_1.true = newlabel()$<br>$B_1.false = B.false$<br>$B_2.true = B.true$<br>$B_2.false = B.false$<br>$B.code = B_1.code \parallel label(B_1.true) \parallel B_2.code$ |
| $B \rightarrow ! B_1$                | $B_1.true = B.false$<br>$B_1.false = B.true$<br>$B.code = B_1.code$                                                                                                  |
| $B \rightarrow E_1 \text{ rel } E_2$ | $B.code = E_1.code \parallel E_2.code$<br>$\parallel gen('if' E_1.addr \text{ rel.op } E_2.addr 'goto' B.true)$<br>$\parallel gen('goto' B.false)$                   |

Traversing the parse tree to evaluate the attributes helps generate intermidate code



# Example

- `if ( x < 100 || x > 200 && x != y ) x = 0;`



Generated code:

```

 if x < 100 goto L2
 goto L3
L3: if x > 200 goto L4
 goto L1
L4: if x != y goto L2
 goto L1
L2: x = 0
L1:

```

使用图 6.36 和 6.37 中的语法制导定义，我们将得到图 6.38 中的代码。

语句 `if ( x < 100 || x > 200 && x != y ) x = 0;` 构成了由  $P \rightarrow S$  生成的程序。这个产生式的语义规则生成了 `S` 的代码之后第一个指令的新标号 `L1` 。

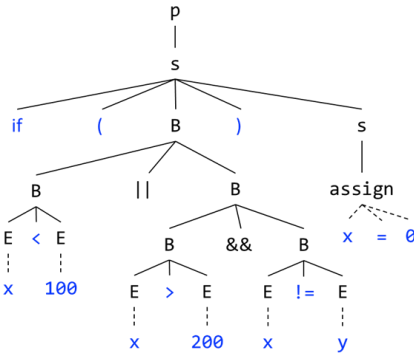
语句 `S` 的形式为 `if(B) S1`，其中 `S1` 是 `x = 0;`，因此，图 6.36 中的规则生成一个新的标号 `L2`，并将其附加到 `S1.code` 中的第一条（也是唯一的，在本例中）指令，即 `x = 0` 处。

因为 `||` 优先级低于 `&&`，所以式(6.13) 中布尔表达式的形式为 `B1 || B2`，其中 `B1` 是 `x < 100`。遵循图 6.37 中的规则，`B1:true` 是 `L2`，即语句 `x = 0` 的标号；`B1:false` 是一个新的标号 `L3`，附加到 `B2` 代码中的第一条指令上。

请注意，生成的代码不是最佳的，因为翻译结果比例 6.21 中的代码多了 3 条指令（`goto`）。指令 `goto L3` 是多余的，因为 `L3` 是下一条指令的标号。如像例 6.21 所示使用 `ifFalse` 代替 `if` 指令，可以消除两条 `goto L1` 指令，。

## Example (Optimization)

- `if ( x < 100 || x > 200 && x != y ) x = 0;`



Optimization: Using `ifFalse` avoids redundant `gotos`

```

if x < 100 goto L2
ifFalse x > 200 goto L1
ifFalse x != y goto L1
L2: x = 0
L1:

```

这个 `ifFalse` 指令利用了控制流会从一个指令自然流动到下一个指令的性质，因此当 `x > 200` 时，控制就直接“穿越”到标号 `L1`，从而减少了一个跳转指令。

## Boolean Values and Jumping Code

- Besides being used to alter the flow of control in statement, a boolean expression may also be evaluated for its value
  - Example: `x = a < b`
- Grammar:
  - `S → id = E ; | if ( E ) S | while ( E ) S | S S`
  - `E → E || E | E && E | E rel E | E + E | ( E ) | id | true | false`
- A way of handling both roles of boolean expressions is to first build a syntax tree for expressions, and then use separate code-generation functions depending on the context of `E`
  - For example, for `E` in `S → while ( E ) S1`, generate jumping code
  - For `E` in `S → id = E`, generate code to compute boolean values

布尔值和跳转代码

布尔表达式除了用于改变语句中的控制流外，还可以求值

示例：  $x = a < b$

语法：

处理布尔表达式两种角色的方法是，首先为表达式建立语法树，然后根据  $E$  的上下文使用不同的代码生成函数。

例如，对于  $S \rightarrow \text{while}(E)S1$  中的  $E$ ，生成跳转代码

对于  $S \rightarrow \text{id} = E$  中的  $E$ ，生成计算布尔值的代码

## Backpatching (回填)\*

- A **key problem** when generating code for boolean expressions and flow-of-control statements is to **match a jump instruction with the jump target**
- **Example:  $\text{if}(B) S$** 
  - According to the short-circuit translation,  $B$ 's code contains a jump to the instruction following the code for  $S$  (executed when  $B$  is false)
  - However,  $B$  must be translated before  $S$ . The jump target is unknown when translating  $B$
  - Earlier, we address the problem by passing labels as inherited attributes ( $S.\text{next}$ ), but this requires another separate pass to bind labels to addresses

How to address the problem in one pass?



\* Will be discussed in lab sessions

在生成布尔表达式和流程控制语句代码时，一个关键问题是将跳转指令与跳转目标匹配起来。

示例：  $\text{if}(B) S$

根据短路翻译，布尔表达式  $B$  的翻译结果中包含一个跳转指令。当  $B$  为假时，该指令将跳转到紧跟在  $S$  的代码之后的指令处。在一趟式的翻译中， $B$  必须在处理  $S$  之前就翻译完毕。但是，在翻译  $B$  时，跳转目标是未知的。

早些时候，我们通过将标号作为继承属性传递到生成相关跳转指令的地方来解决这个问题(*S. next*)，但这需要再进行一趟处理，将标号和具体的地址绑定。

# One-Pass Code Generation Using Backpatching (1)

- **Basic idea of backpatching (基本思想):**
  - When a jump is generated, its target is temporarily left unspecified
  - Incomplete jumps are grouped into lists. All jumps on a list have the same target
  - Fill in the labels for incomplete jumps when the targets become known
- **The technique (技术细节):**
  - For a nonterminal  $B$  that represents a boolean expression, we define two synthesized attributes: *truelist* and *falselist*
  - *truelist*: a list of jump instructions for which we must fill in the label to which control goes if  $B$  is true
  - *falselist*: a list of jump instructions for which we must fill in the label to which control goes if  $B$  is false

本节采用一种称为回填的补充方法，它把一个由跳转指令组成的列表以综合属性的形式进行传递。具体来说，当生成一个跳转指令时，暂时不指定跳转的目标。这样的跳转都被放在一个跳转列表中，当可以确定正确的目标标号时，才去填充这些指令的目标标号。同一列表中所有跳转指令具有相同的目标标号。

技术细节：

对于表示布尔表达式的非终结符 $B$ ，我们定义了两个综合属性：*truelist* 和 *falselist* 用于管理布尔表达式的跳转代码中的标号。

特别是， $B.truelist$  将是一个包含跳转或条件跳转指令列表，我们必须向这些指令中插入适当的标号，也就是当  $B$  为真时控制流应该转向的标号。类似地， $B.falselist$  也是一个包含跳转指令的列表，这些指令最终获得的标号就是当  $B$  为 *false* 时，控制流应该转向的标号。在生成  $B$  的代码时，跳转到 *true* 和 *false* 出口的跳转指令是不完整的，标号字段尚未填写。这些不完整的跳转指令被保存在  $B.truelist$  和  $B.falselist$  指向的列表中。类似

地，语句  $S$  具有综合属性  $S:\text{nextlist}$  也是一个跳转指令列表，这些指令应该跳转到紧跟在  $S$  的代码之后的指令。

## One-Pass Code Generation Using Backpatching (2)

- **The technique (技术细节) Cont.:**
  - *makelist( $i$ )*: create a new list containing only  $i$ , the index of a jump instruction, and return the pointer to the list
  - *merge( $p_1, p_2$ )*: concatenate the lists pointed by  $p_1$  and  $p_2$ , and return a pointer to the concatenated list
  - *backpatch( $p, i$ )*: insert  $i$  as the target label for each of the instructions on the list pointed by  $p$

我们将生成的指令放入一个指令数组中，而标号将是该数组的下标。为了处理跳转指令的列表，我们使用三个函数：

*makelist( $i$ )*

创建一个只包含  $i$  的列表， $i$  是跳转指令的标号，函数返回指向新创建的列表的指针

*merge( $p_1, p_2$ )*

将  $p_1$  和  $p_2$  指向的列表进行合并，返回指向合并后的列表的指针

*backpatch( $p, i$ )*

将  $i$  作为目标标号插入到  $p$  所指列表中的各指令中

# Backpatching for Boolean Expressions (布尔表达式的回填)

- An SDT suitable for generating code for boolean expressions during bottom-up parsing
- Grammar:
  - $B \rightarrow B_1 \parallel MB_2 \mid B_1 \&\& MB_2 \mid !B_1 \mid (B_1) \mid E_1 \text{ rel } E_2 \mid \text{true} \mid \text{false}$
  - $M \rightarrow \epsilon$

我们现在构建一个可以在自下而上语法分析过程中生成布尔表达式目标代码的翻译方案。这个语法中的标记非终结符  $M$ ，它引发的语义动作在适当的时刻获取将要生成的下一条指令的下标。语法如下：

- 1)  $B \rightarrow B_1 \parallel M B_2$  {  $backpatch(B_1.falselist, M.instr);$   
 $B.truelist = merge(B_1.truelist, B_2.truelist);$   
 $B.falselist = B_2.falselist; \}$
- 2)  $B \rightarrow B_1 \&\& M B_2$  {  $backpatch(B_1.truelist, M.instr);$   
 $B.truelist = B_2.truelist;$   
 $B.falselist = merge(B_1.falselist, B_2.falselist); \}$
- 3)  $B \rightarrow ! B_1$  {  $B.truelist = B_1.falselist;$   
 $B.falselist = B_1.truelist; \}$
- 4)  $B \rightarrow ( B_1 )$  {  $B.truelist = B_1.truelist;$   
 $B.falselist = B_1.falselist; \}$
- 5)  $B \rightarrow E_1 \text{ rel } E_2$  {  $B.truelist = makelist(nextinstr);$   
 $B.falselist = makelist(nextinstr + 1);$   
 $gen('if' E_1.addr \text{ rel.op } E_2.addr 'goto -');$   
 $gen('goto -');$  }
- 6)  $B \rightarrow \text{true}$  {  $B.truelist = makelist(nextinstr);$   
 $gen('goto -');$  }
- 7)  $B \rightarrow \text{false}$  {  $B.falselist = makelist(nextinstr);$   
 $gen('goto -');$  }
- 8)  $M \rightarrow \epsilon$  {  $M.instr = nextinstr; \}$

考虑上述文法中对应于规则 (1)  $B \rightarrow B_1 \parallel M B_2$  的语义动作。如果  $B_1$  为 `true`，则  $B$  也为 `true`，因此  $B_1.truelist$  中的跳转指令就成为  $B.truelist$  的一部分。然而，如果  $B_1$  为假，我们接下来必须测试  $B_2$ 。因此  $B_1.falselist$  中的跳转指令的目标必定  $B_2$  的代码的起始位置。这个位置使用标记非终结符  $M$  获得。在即将生成  $B_2$  代码之前， $M$  生成了下一条指令的标号，存放在综合属性  $M.instr$  中。

为了获得该指令标号，我们将产生式  $M \rightarrow \epsilon$  和语义动作动作  $\{M.instr = nextinstr; \}$  关联起来。变量 `nextinstr` 保存了紧跟着的下一条指令的标号。当我们看到产生式  $B \rightarrow B_1 \parallel M B_2$  的剩余部分时，该值将被回填到  $B_1.falselist$  中的指令上（即列表  $B_1.falselist$  上的每条指令都将接收  $M.instr$  作为其目标标号）。

$B \rightarrow B_1 \parallel M B_2$  的语义动作 (2) 和  $B \rightarrow B_1 \&\& M B_2$  语义动作(1)类似。  $B \rightarrow ! B$  的语义动作 (3) 交换真假列表。动作 (4)只是 忽略括号。

为简单起见，语义动作 (5) 生成两条指令，一条有条件转移指令 `goto` 和一条无条件指令。两者的目标标号都没有填充。这些指令被放入新列表中，分别由  $B.truelist$  和  $B.falselist$  指向。

# Backpatching vs. Non-Backpatching (1)

(1) Non-backpatching SDD:

|                                      |                                                                                                                                                                             |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $B \rightarrow E_1 \text{ rel } E_2$ | $B.code = E_1.code \parallel E_2.code$<br>$\parallel \text{gen('if' } E_1.addr \text{ rel.op } E_2.addr \text{ 'goto' } B.true)$<br>$\parallel \text{gen('goto' } B.false)$ |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

(2) Backpatching scheme:

|                                      |                                                                                                                                                                                                     |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $B \rightarrow E_1 \text{ rel } E_2$ | $\{$ $B.truelist = makelist(nextinstr);$<br>$B.falselist = makelist(nextinstr + 1);$<br>$\text{gen('if' } E_1.addr \text{ rel.op } E_2.addr \text{ 'goto' } \_);$<br>$\text{gen('goto' } \_);$ $\}$ |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Comparison:

- In (2), incomplete instructions (指令坯) are added to corresponding lists
- The instruction jumping to  $B.true$  in (1) is added to  $B.truelist$  in (2)
- The instruction jumping to  $B.false$  in (1) is added to  $B.falselist$  in (2)

比较:

在 (2) 中, 指令坯被添加到相应的列表中。

跳转到  $B$  的指令。(1) 中的  $B.true$ 被添加到 (2)中的 $B.truelist$ 。

跳转到  $B$  的指令。(1)中的 $B.false$  被添加到 (2) 中的 $B.falselit$



## Backpatching vs. Non-Backpatching (2)

(1) Non-backpatching SDD:

|                                   |                                                                                                                                                                      |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $B \rightarrow B_1 \parallel B_2$ | $B_1.true = B.true$<br>$B_1.false = newlabel()$<br>$B_2.true = B.true$<br>$B_2.false = B.false$<br>$B.code = B_1.code \parallel label(B_1.false) \parallel B_2.code$ |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|

(2) Backpatching scheme:

|                                     |                                                                                                                                     |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| $B \rightarrow B_1 \parallel M B_2$ | $\{$ $backpatch(B_1.falselist, M.instr);$<br>$B.truelist = merge(B_1.truelist, B_2.truelist);$<br>$B.falselist = B_2.falselist; \}$ |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|

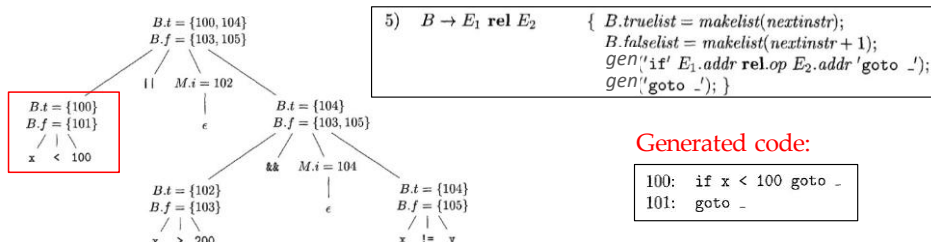
### Comparison:

- The assignments to *true/false* attributes in (1) correspond to the assignments to *truelist/falselist* or *merge* in (2)

(1) 中对 *true/false* 属性的赋值与 (2) 中对 *truelist/falselist* 或 *merge* 属性的赋值相对应。

## Example – Boolean Expressions (1)

- The earlier SDT is a **postfix SDT**. The semantic actions can be performed during a bottom-up parse.
- Boolean expression:  $x < 100 \parallel x > 200 \ \&\& \ x \neq y$
- Step 1:** reduce  $x < 100$  to  $B$  by production (5)

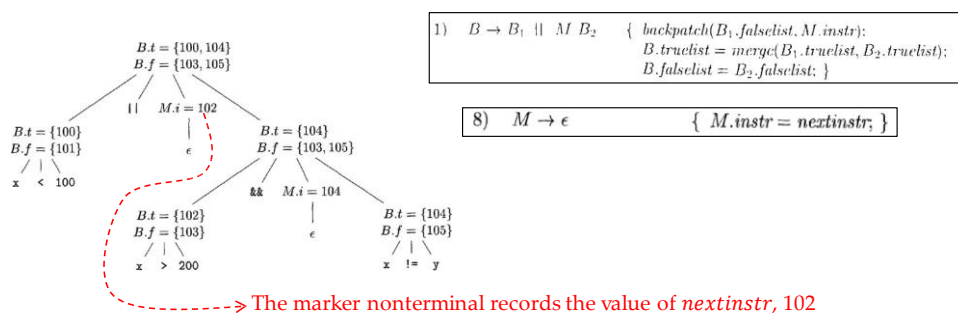


带注释的语法分析树如图 6.44 所示；为了便于阅读，属性 `truelist`、`falselist` 和 `instr` 由其首字母表示。这些动作是在树的深度优先遍历期间执行的。由于所有动作都出现在产生式右侧的末尾，因此它们可以在自下而上的语法分析过程中与归约一起执行。在根据产生式 (5) 将  $x < 100$  归约为  $B$  时，语义动作相应地产生两条指令：

我们可任意从 100 开始为指令编号。

## Example – Boolean Expressions (1)

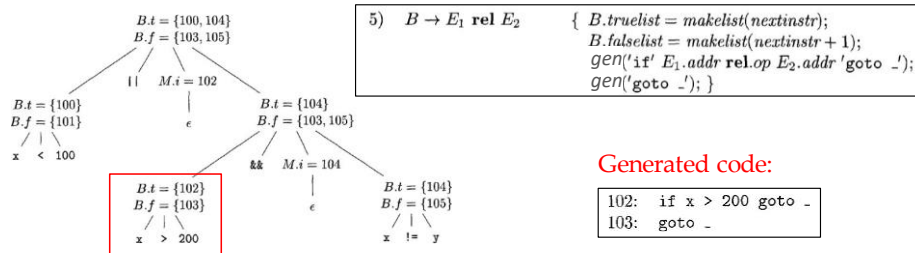
- The earlier SDT is a **postfix SDT**. The semantic actions can be performed during a bottom-up parse.
- Boolean expression:  $x < 100 \parallel x > 200 \&\& x \neq y$
- **Step 2:** reduce  $\epsilon$  to  $M$  by production (8)



产生式 (8) 中的标记非终结符记录下一条指令的标号 102.

## Example – Boolean Expressions (2)

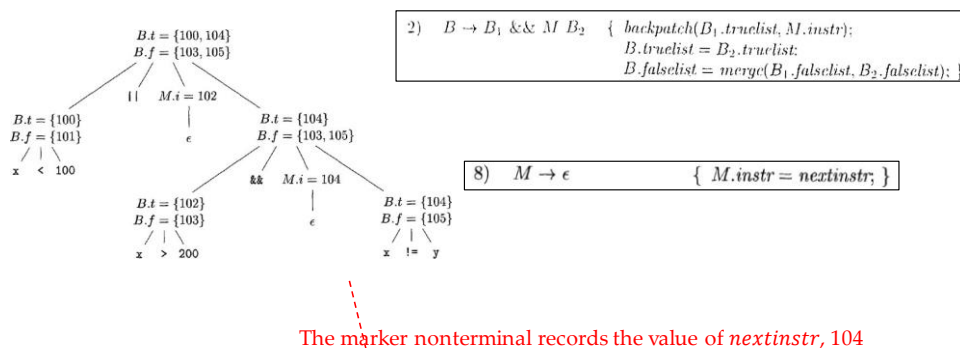
- Boolean expression:  $x < 100 \parallel x > 200 \ \&\& \ x \neq y$
- **Step 3:** reduce  $x > 200$  to  $B$  by production (5)



根据产生式 (5) 将  $x > 200$  归约为 B 时, 产生两条指令:

## Example – Boolean Expressions (2)

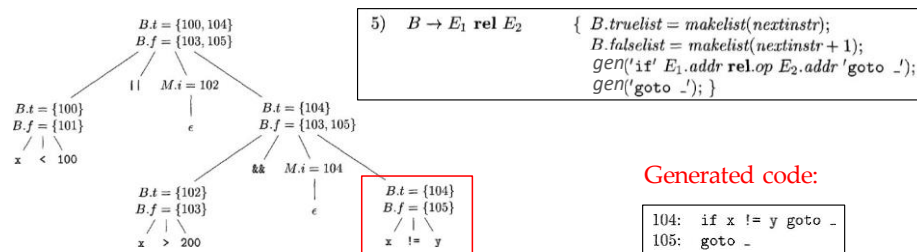
- Boolean expression:  $x < 100 \parallel x > 200 \ \&\& \textcolor{red}{x} \neq \textcolor{red}{y}$
- **Step 4:** reduce  $\epsilon$  to  $M$  by production (8)



子表达式  $x > 200$  对应于产生式 (2) 中的 B1, 标记非终结符 M 记录了下一条指令的标号 104

## Example – Boolean Expressions (3)

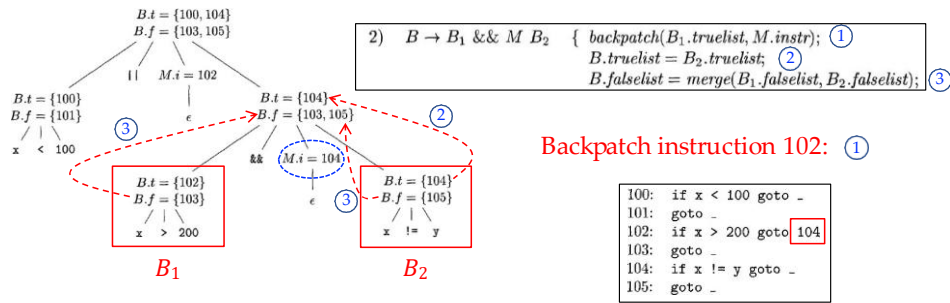
- Boolean expression:  $x < 100 \parallel x > 200 \ \&\& \ x \neq y$
- **Step 5:** reduce  $x \neq y$  to B by production (5)



根据产生式 (5) 将  $x = y$  归约为 B 时, 产生两条指令:

## Example – Boolean Expressions (4)

- Boolean expression:  $x < 100 \parallel x > 200 \&\& x \neq y$
- **Step 6:** reduce  $B_1 \&\& M B_2$  to  $B$  by production (2)

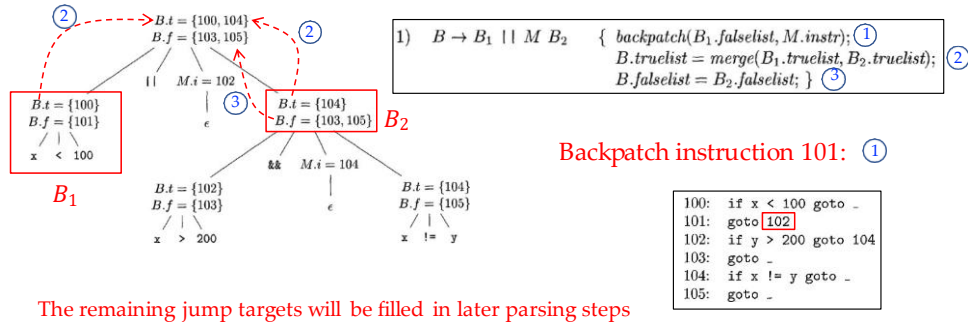


我们现在使用产生式 (2)  $B \rightarrow B_1 \&\& M B_2$  进行归约。相应的语义动作调用  $\text{backpatch}(B_1:\text{truelist}; M:\text{instr})$  将  $B_1$  的真出口绑定到  $B_2$  的第一条指令处。由于  $B_1:\text{truelist}$  为  $\{102\}$ ,  $M:\text{instr}$  为 104, 因此此次对  $\text{backpatch}$  的调用将序号 104 填写到 102 指令中。到目前为止生成的 6 个指令如图 6.45(a) 所示。

1.  $M.\text{instr} = 104 \rightarrow B_1.t\{102\}$
2.  $B.\text{truelist} = B_2.\text{truelist}\{104\}$
3.  $B.\text{falselist} = \text{merge}(B_1.\text{falselist}\{103\}, B_2.\text{falselist}\{105\})$

## Example – Boolean Expressions (5)

- Boolean expression:  $x < 100 \parallel x > 200 \ \&\& \ x \neq y$
- **Step 7:** reduce  $B_1 \parallel MB_2$  to  $B$  by production (1)



与产生式(1)  $B \rightarrow B_1 \parallel M B_2$  相关联的语义动作调用  $\text{backpatch}(\{101\}, 102)$ ，得到如图 6.45(b) 所示的指令。整个表达式为 true 当且仅当控制流到达 100 或 104 位置上的跳转指令；整个表达式为 false 当且仅当控制流到达 103 或 105 位置上的跳转指令。在稍后的编译中，当已知表达式为真或假时分别应该做什么的时候，这些指令的目标将会被填写完整

1.  $\text{backpatch}(B_1.\text{falselist}\{101\}, M.\text{instr}\{102\})$
2.  $B.\text{truelist} = \text{merge}(B_1.\text{truelist}\{100\}, B_2.\text{truelist}\{104\}) = \{100, 104\}$
3.  $B.\text{falselist} = B_2.\text{falselist}\{103, 105\} = \{103, 105\}$